

Crash Course of Deep Learning Systems

Foundation to Optimization

2025/07/06

@erix025

ZJUSCT

Contents

- Foundation of Deep Learning
- **Topics in MLSys Optimization**

Contents – Topics in MLSys Optimization

- Hardware in Deep Learning
- Pruning
- Quantization
- Parallelism
- Other Topics

Hardware in Deep Learning

CPU vs GPU

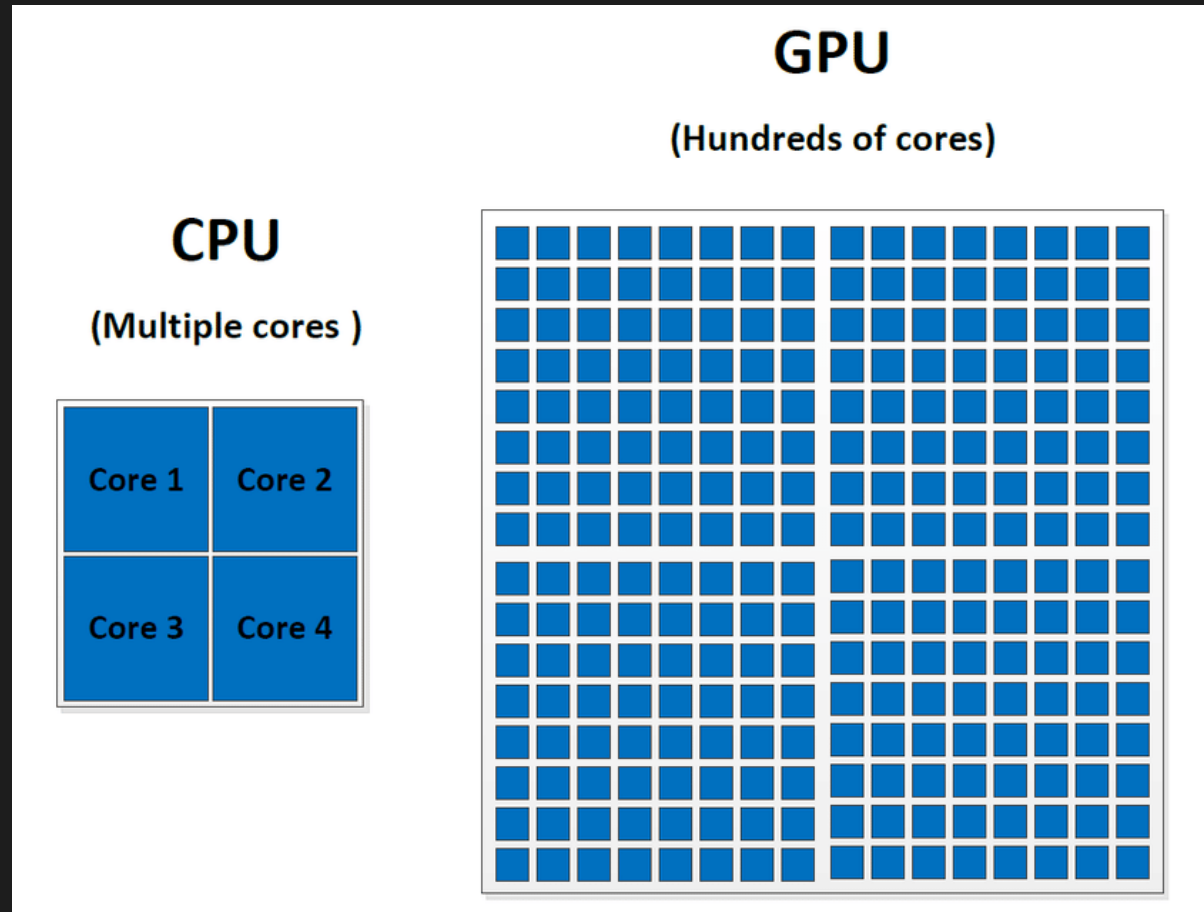
CPU: Fewer cores, but each core is much faster and much more capable; great at sequential tasks

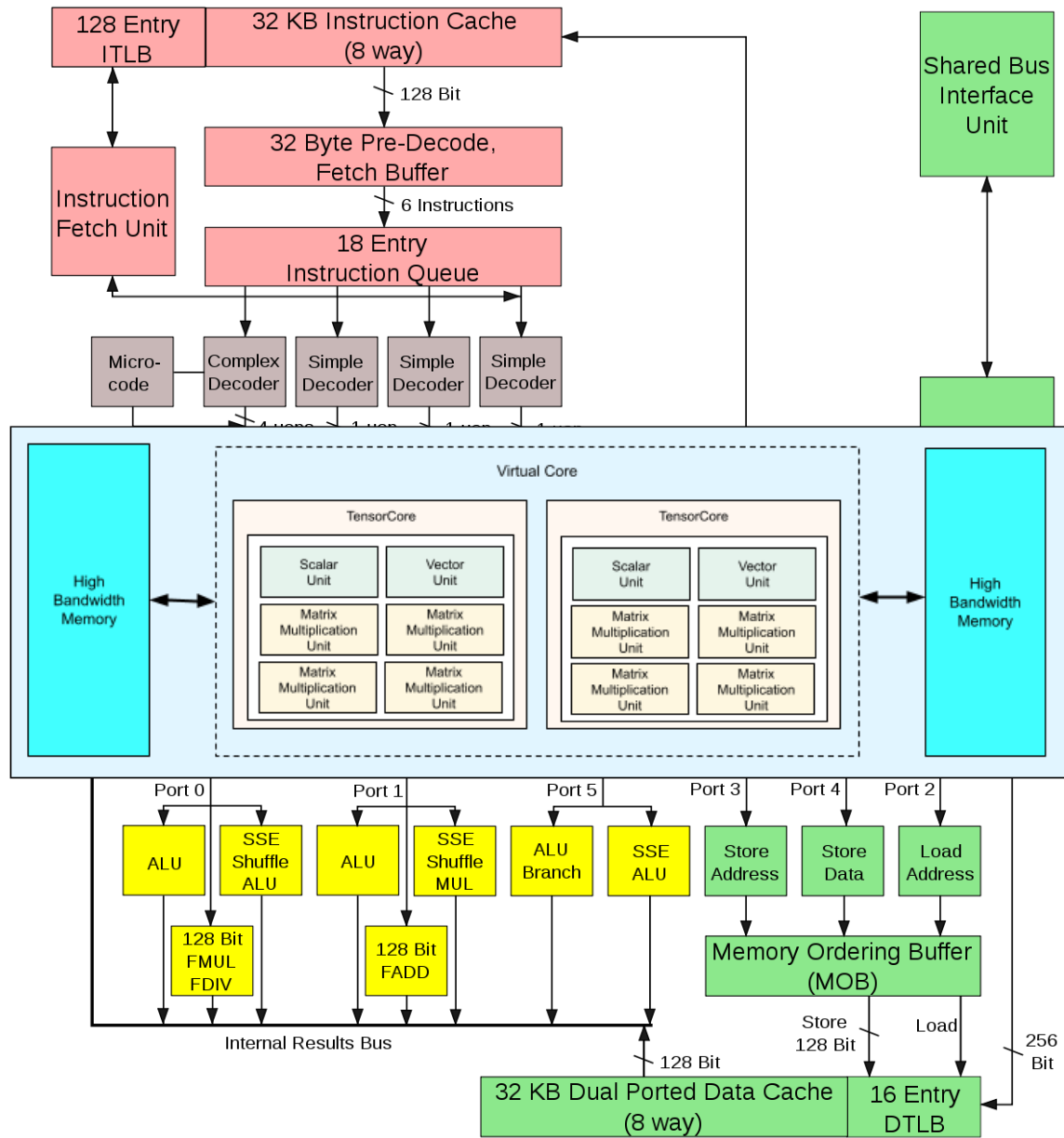
GPU: More cores, but each core is much slower and “dumber”; great for parallel tasks

TPU: Specialized hardware for deep learning

	Cores	Clock Speed	Memory	Price	Compute Capability
CPU Intel® Xeon® Platinum 8558 Processor	48	2.1GHz	System RAM	\$4,650	6.45TFlops(FP32)
GPU NVIDIA RTX 4090	16384 Cuda Cores 512 Tensor Cores	2.2GHz	24GB GDDR6X	\$1,600	82.5 TFlops(FP32)
GPU NVIDIA Hopper H100 PCIe	14592 Cuda Cores 456 Tensor Cores	1.6GHz	80GB HBM2e	\$25,000	756TFlops(FP32 TensorCore)
TPU Google Cloud TPU v5e	4 Matrix Units(MXU) per core	?	16 GBHBM2	\$1.2(per hour)	197TFlops(BF16)

CPU vs GPU

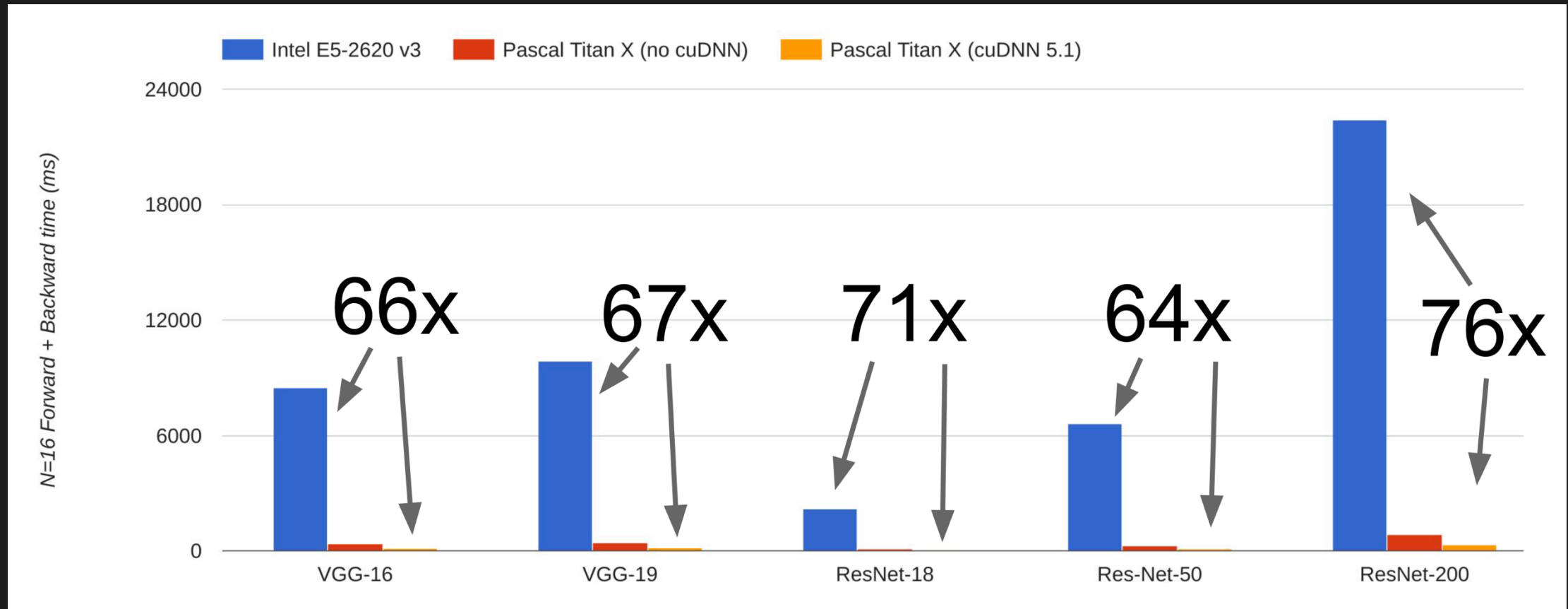




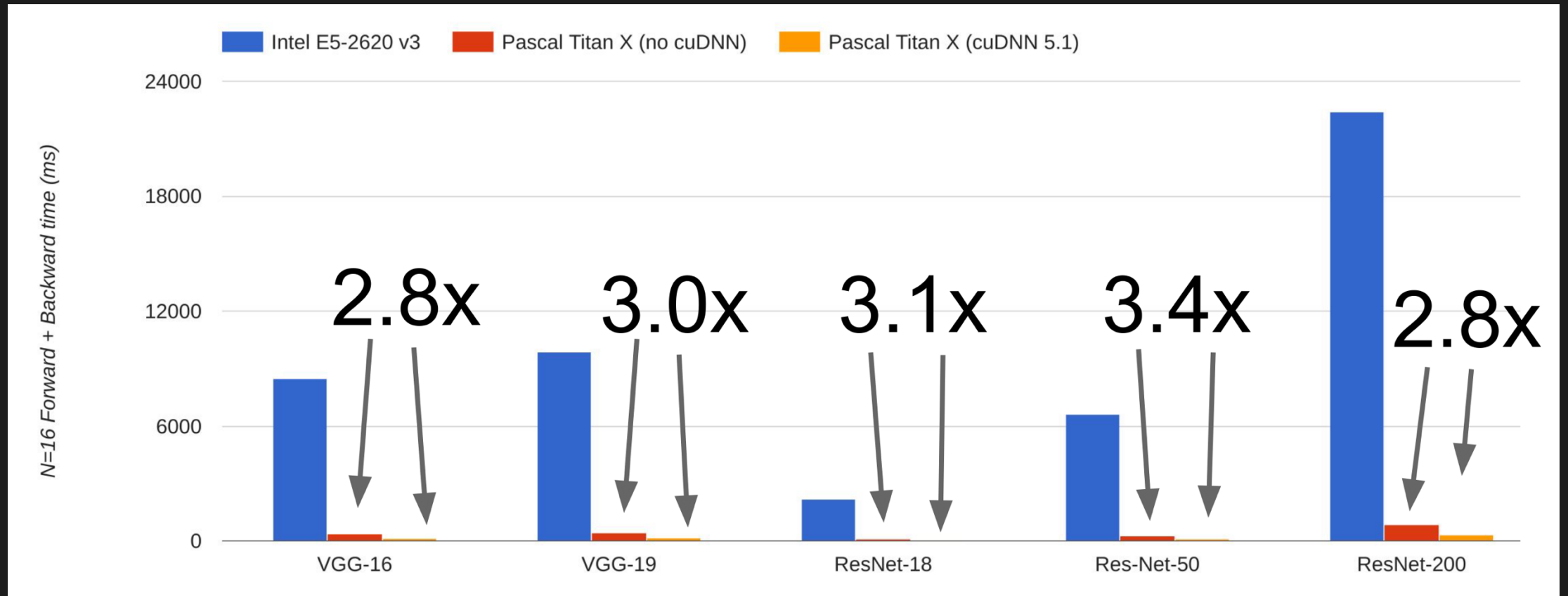
Intel Core 2 Architecture



CPU vs GPU



Plain CUDA vs cuDNN

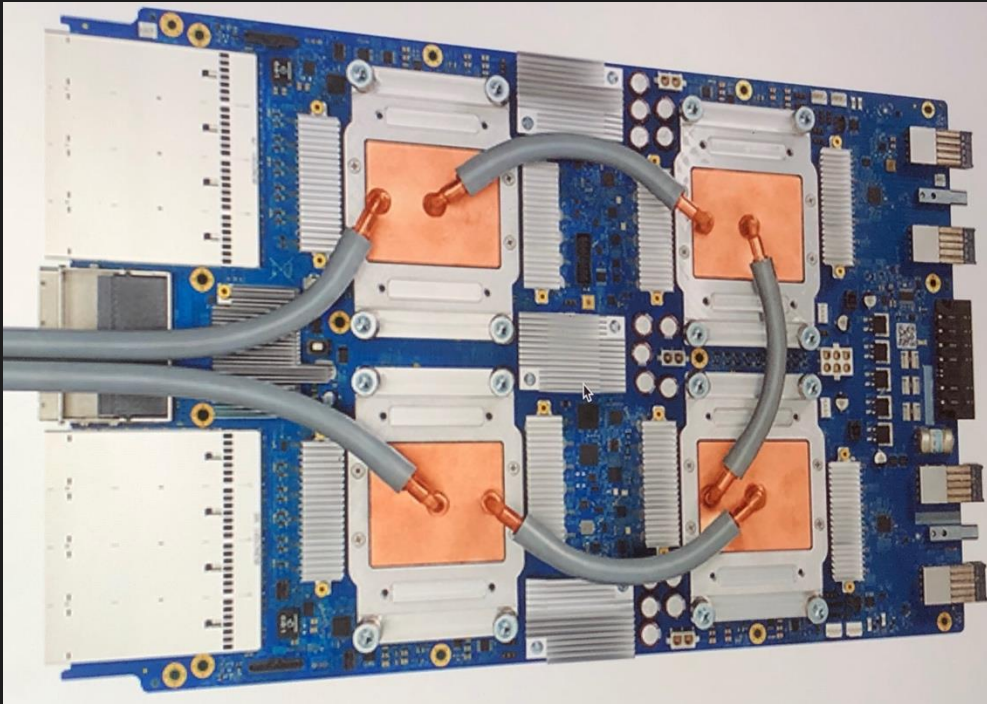


Programming GPUs

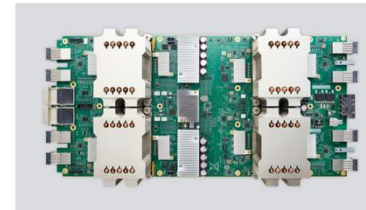
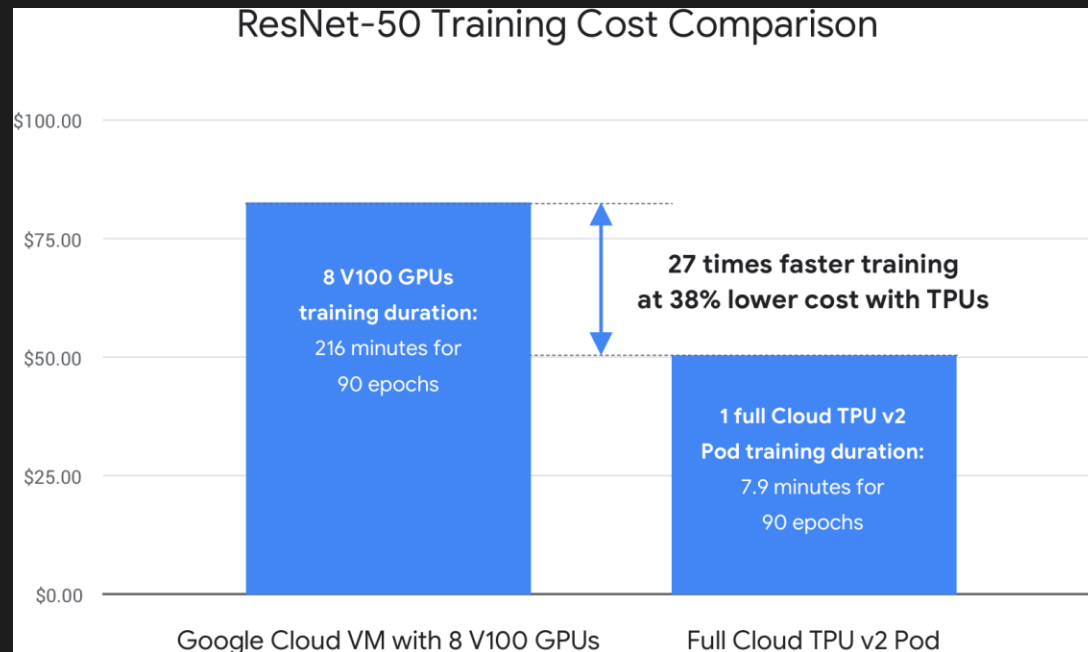
- CUDA
 - Covered in previous sessions
 - Optimized APIs: cuBLAS, cuFFT, cuDNN, etc
- OpenCL
 - Similar to CUDA, but runs on almost anything
 - Usually slower on NVIDIA hardware (30% according to Fang et al.)
- AMD ROCm / HIP



Tensor Processing Unit (TPU)



Tensor Processing Unit (TPU)



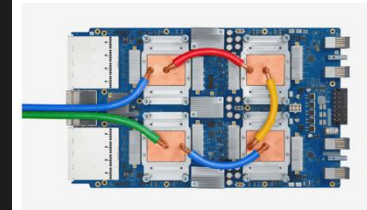
Cloud TPU v2

180 teraflops
64 GB High Bandwidth Memory (HBM)



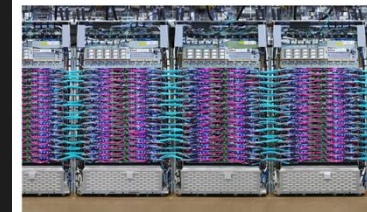
Cloud TPU v2 Pod

11.5 petaflops
4 TB HBM
2-D toroidal mesh network



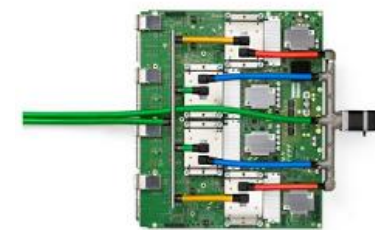
Cloud TPU v3

420 teraflops
128 GB HBM



Cloud TPU v3 Pod

100+ petaflops
32 TB HBM
2-D toroidal mesh network



TPU v4

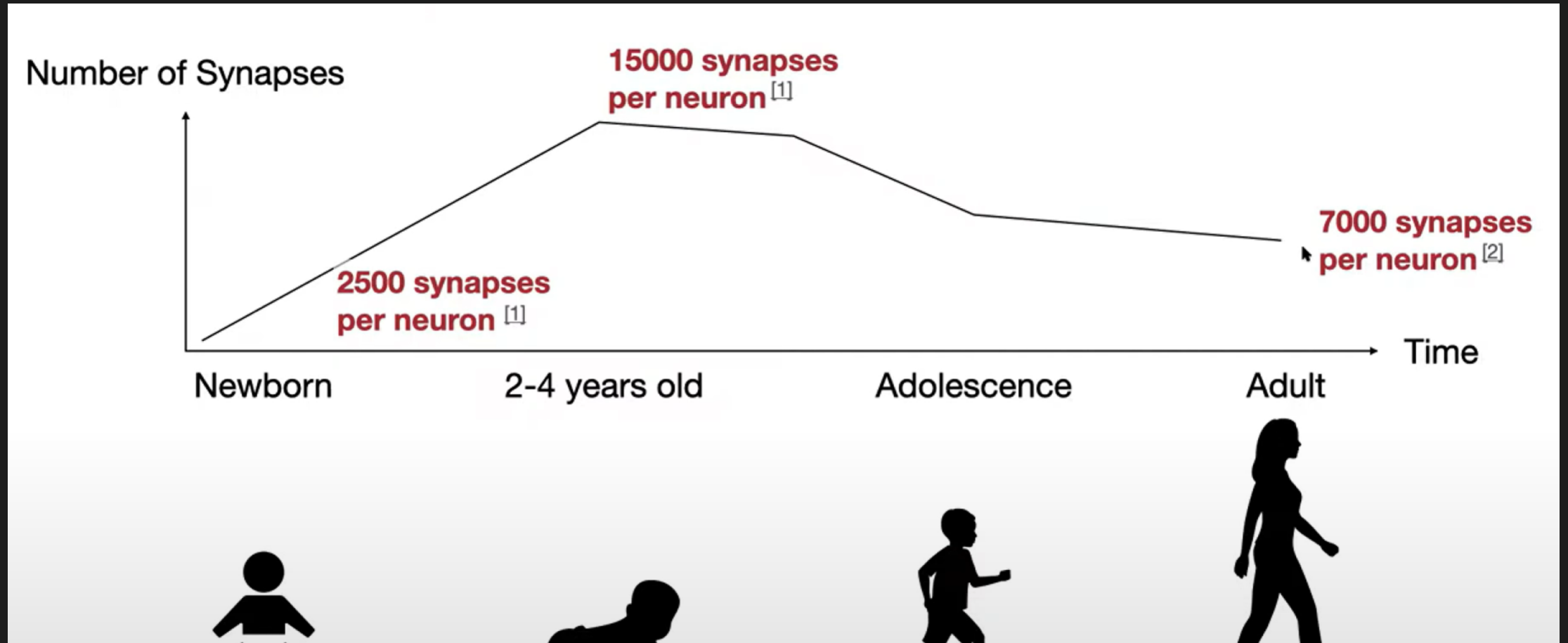


TPU v4 Pod

Any Questions?

Pruning

What is Pruning



What is Pruning

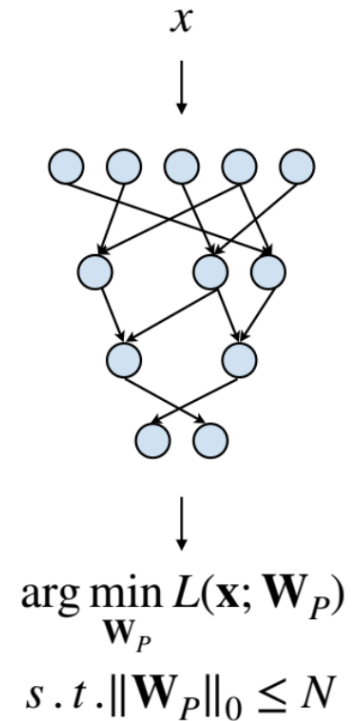
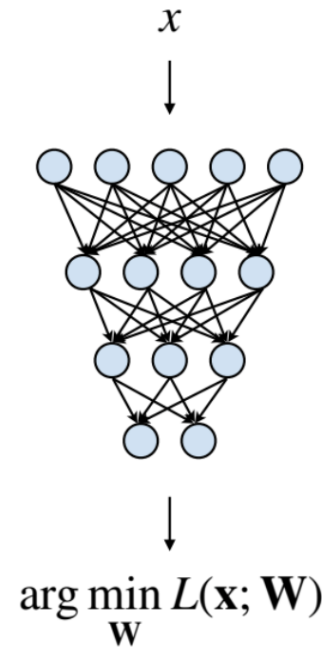
- In general, we could formulate the pruning as follows:

$$\arg \min_{\mathbf{W}_p} L(\mathbf{x}; \mathbf{W}_p)$$

subject to

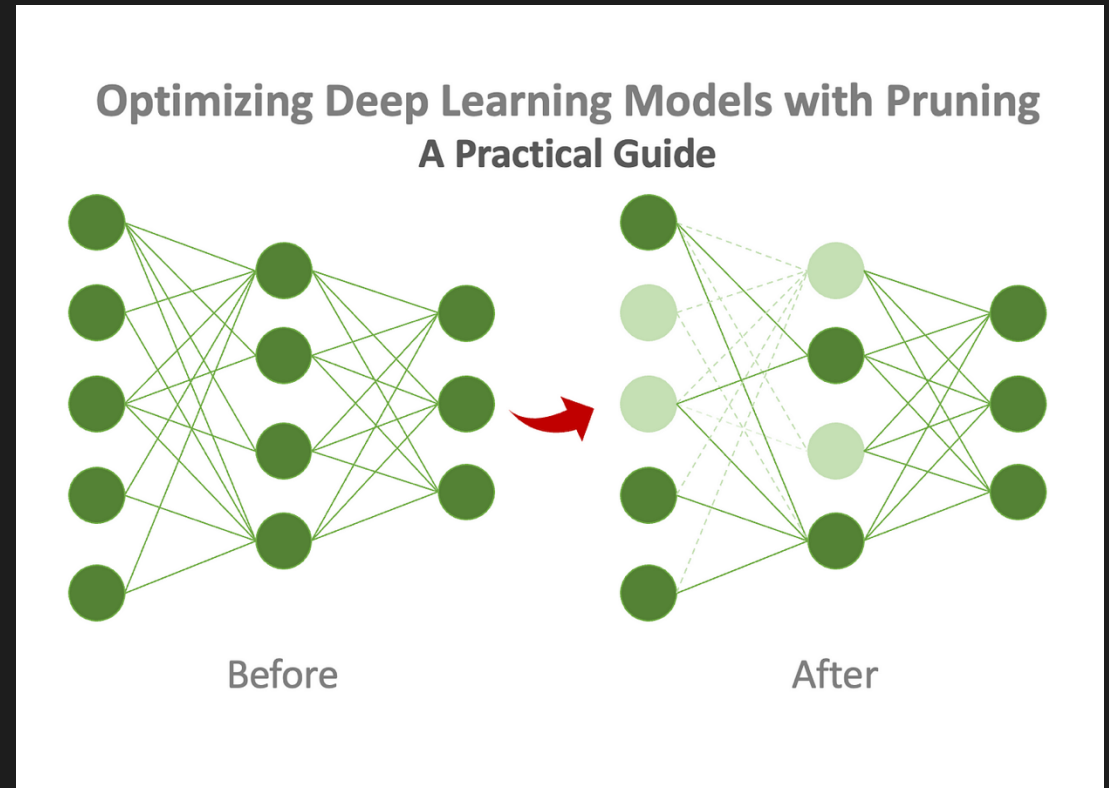
$$\|\mathbf{W}_p\|_0 \leq N$$

- L represents the objective function for neural network training;
- $\mathbf{x}$ is input, $\mathbf{W}$ is original weights, $\mathbf{W}_p$ is pruned weights;
- $\|\mathbf{W}_p\|_0$ calculates the #nonzeros in $\mathbf{W}_p$, and N is the target #nonzeros.



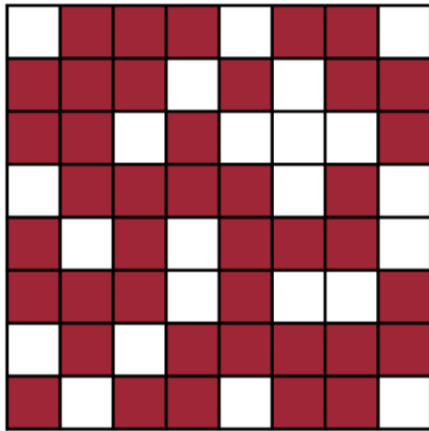
Why We Need Pruning

- Models are over-parameterized
- Some parameters are unimportant
- Reduce computation and memory usage



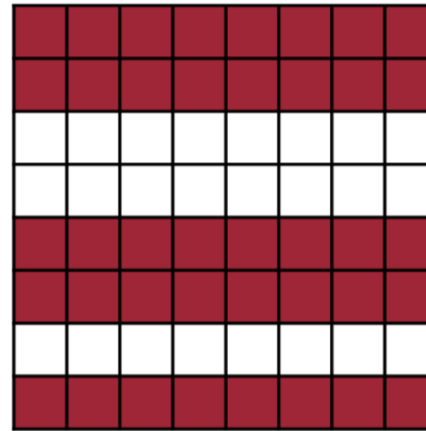
Pruning in Weight -> Sparsity

A simple example of 2D weight matrix



Fine-grained/Unstructured

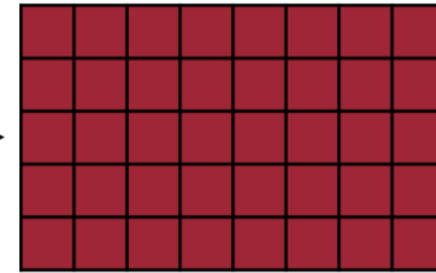
- More flexible pruning index choice
- Hard to accelerate (irregular)



Coarse-grained/Structured

- Less flexible pruning index choice (a subset of the fine-grained case)
- Easy to accelerate (just a smaller matrix!)

 Preserved
 Pruned



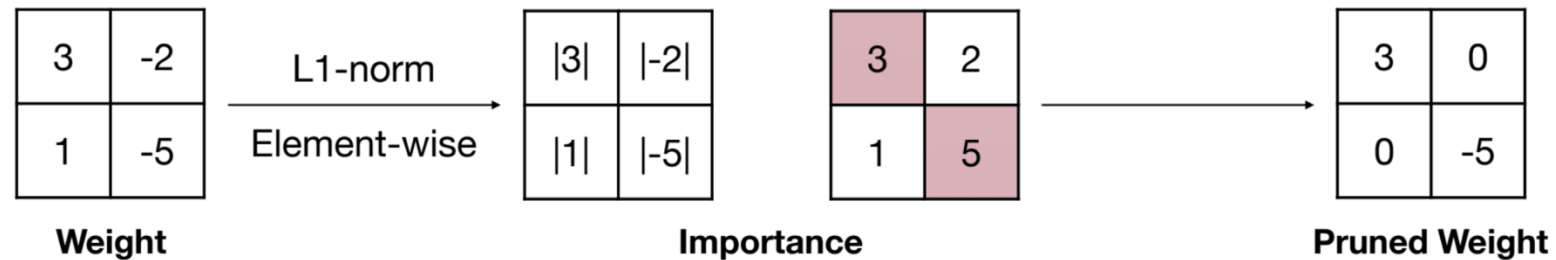
How to Prune

Based on Magnitude

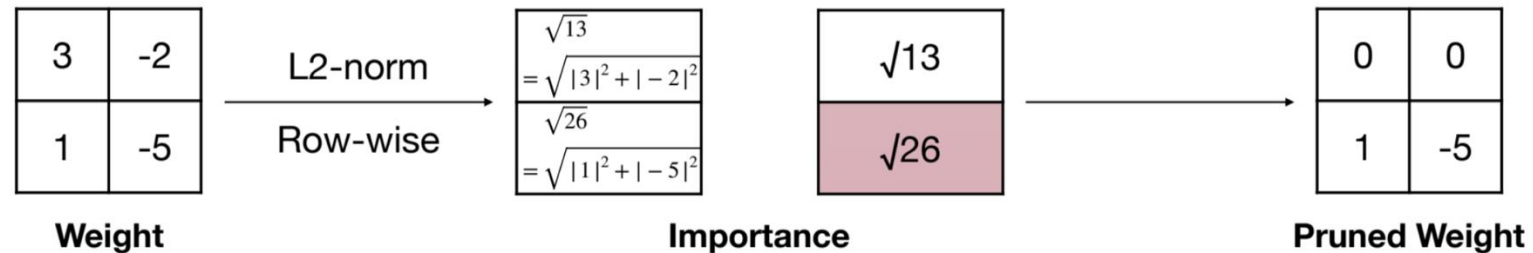
- For element-wise pruning,

$$\text{Importance} = |W|$$

- Example**



- Example**



How to Prune

Based on Loss Minimization

$$\delta L = L(\mathbf{x}; \mathbf{W}) - L(\mathbf{x}; \mathbf{W}_P = \mathbf{W} - \delta \mathbf{W}) = \sum_i \cancel{g_i w_i} + \frac{1}{2} \sum_i h_{ii} \delta w_i^2 + \frac{1}{2} \sum_{i \neq j} \cancel{h_{ij} w_i \delta w_j} + \cancel{O(\|\mathbf{W}\|^3)}$$

where

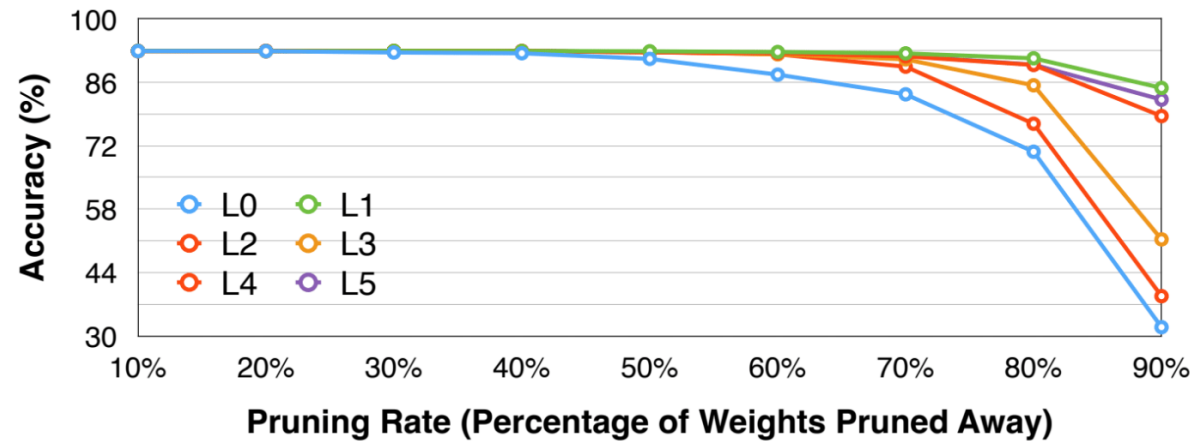
$$g_i = \frac{\partial L}{\partial w_i}, h_{i,j} = \frac{\partial^2 L}{\partial w_i \partial w_j}$$

- Optimal Brain Damage assumes that
 - The objective function L is nearly quadratic: the last term is neglected
 - The neural network training has converged: first-order terms are neglected
 - The error caused by deleting each parameter is independent: cross terms are neglected

$$\delta L_i = L(\mathbf{x}; \mathbf{W}) - L(\mathbf{x}; \mathbf{W}_P | w_i = 0) \approx \frac{1}{2} h_{ii} w_i^2$$

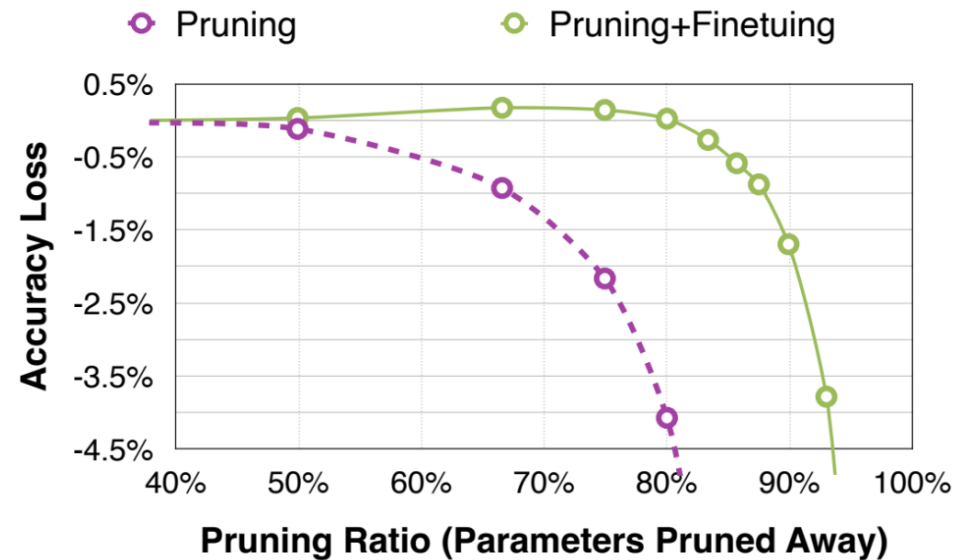
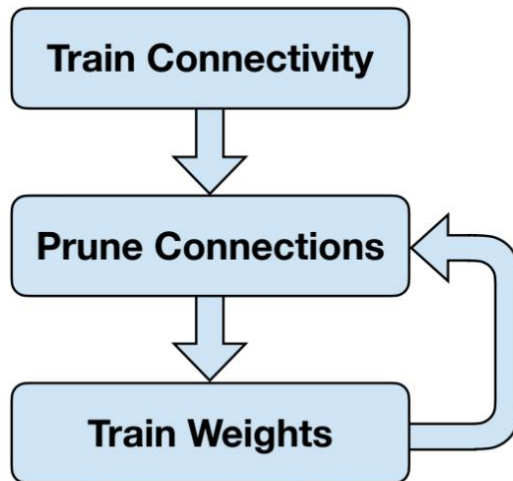
How to Determine Sparsity Ratio

- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
 - Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
 - Repeat the process for all layers



Pruning + Finetuning

- After pruning, the model may decrease, especially for larger pruning ratio.
- Fine-tuning the pruned neural networks will help recover the accuracy and push the pruning ratio higher.
 - **Learning rate for fine-tuning is usually 1/100 or 1/10 of the original learning rate.**

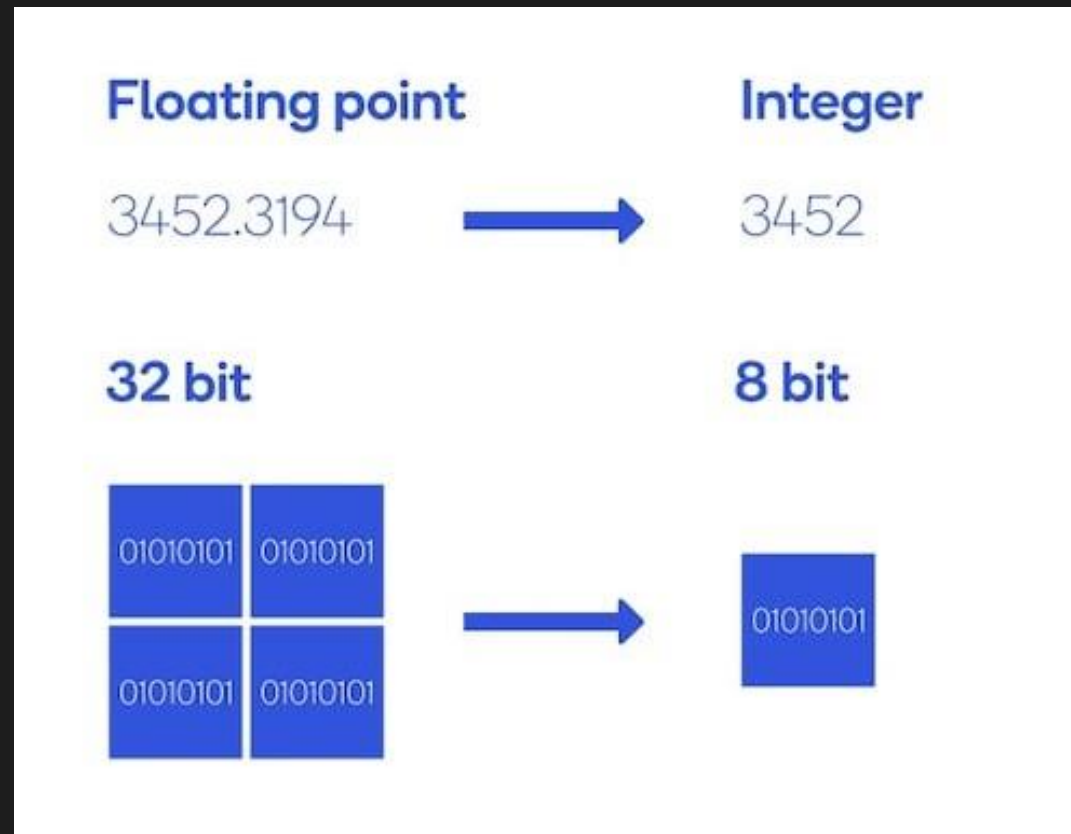


Any Questions?

Quantization

What is Quantization

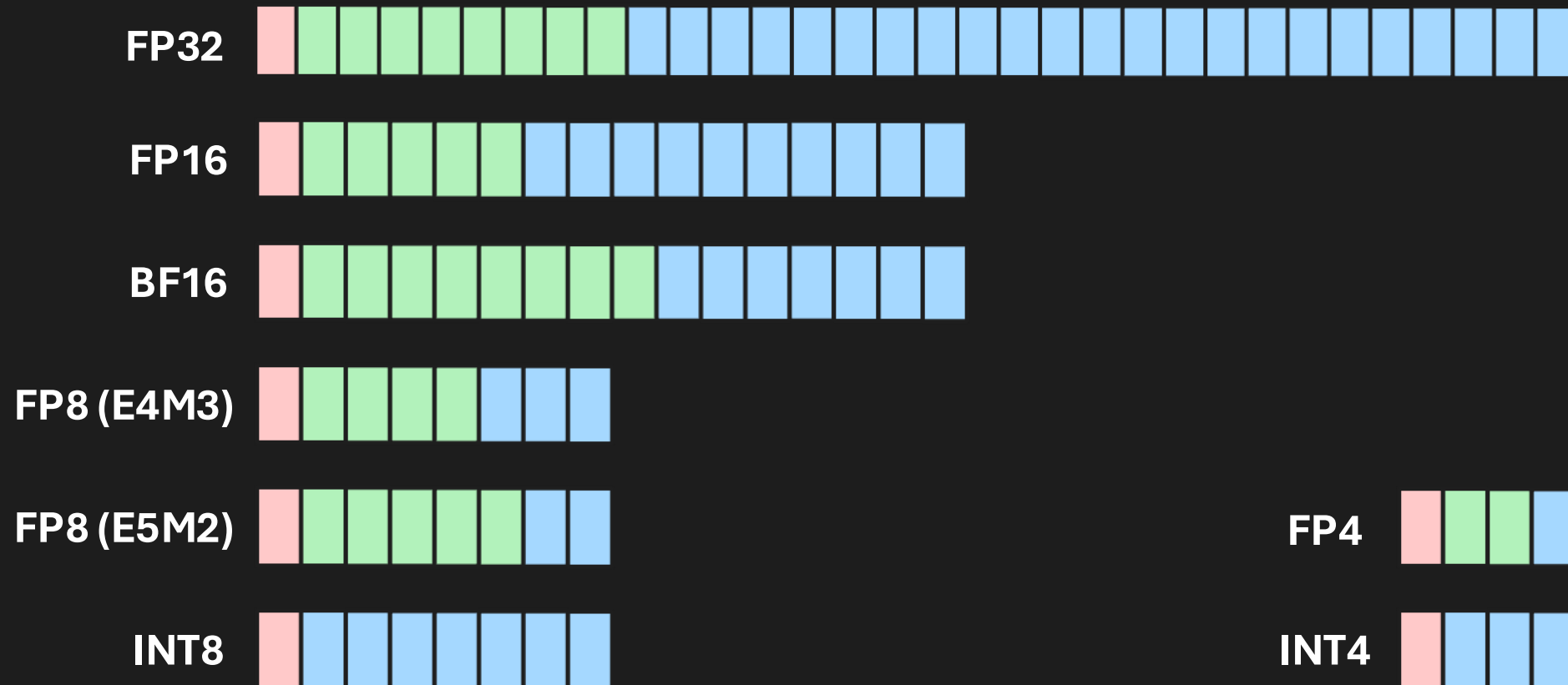
Represent a number in lower precision data type



Why We Need Quantization

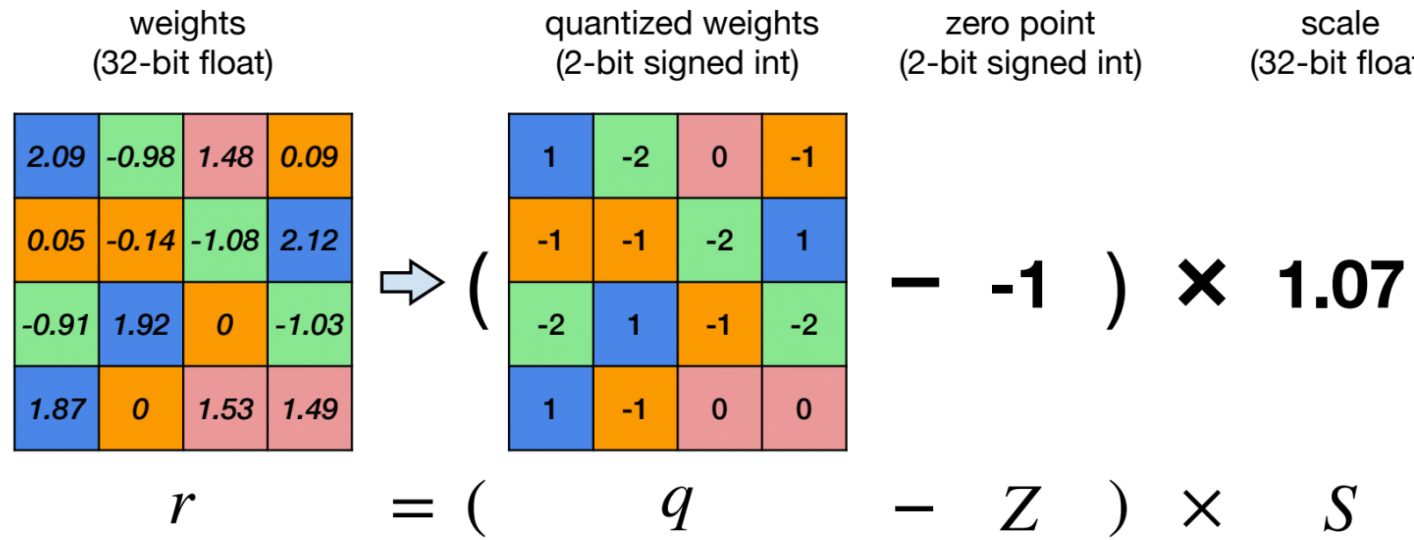
- Pruning is not so good
 - Fine-grained sparsity are not hardware friendly
 - Computation speedup needs special hardware support
 - Hard to reduce memory usage
- Matrix after quantization is still dense
 - Easy to reduce memory usage
 - Still a dense GEMM (in lower precision)

Low Precision Data Type



How to Perform Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$



Floating-point

Integer

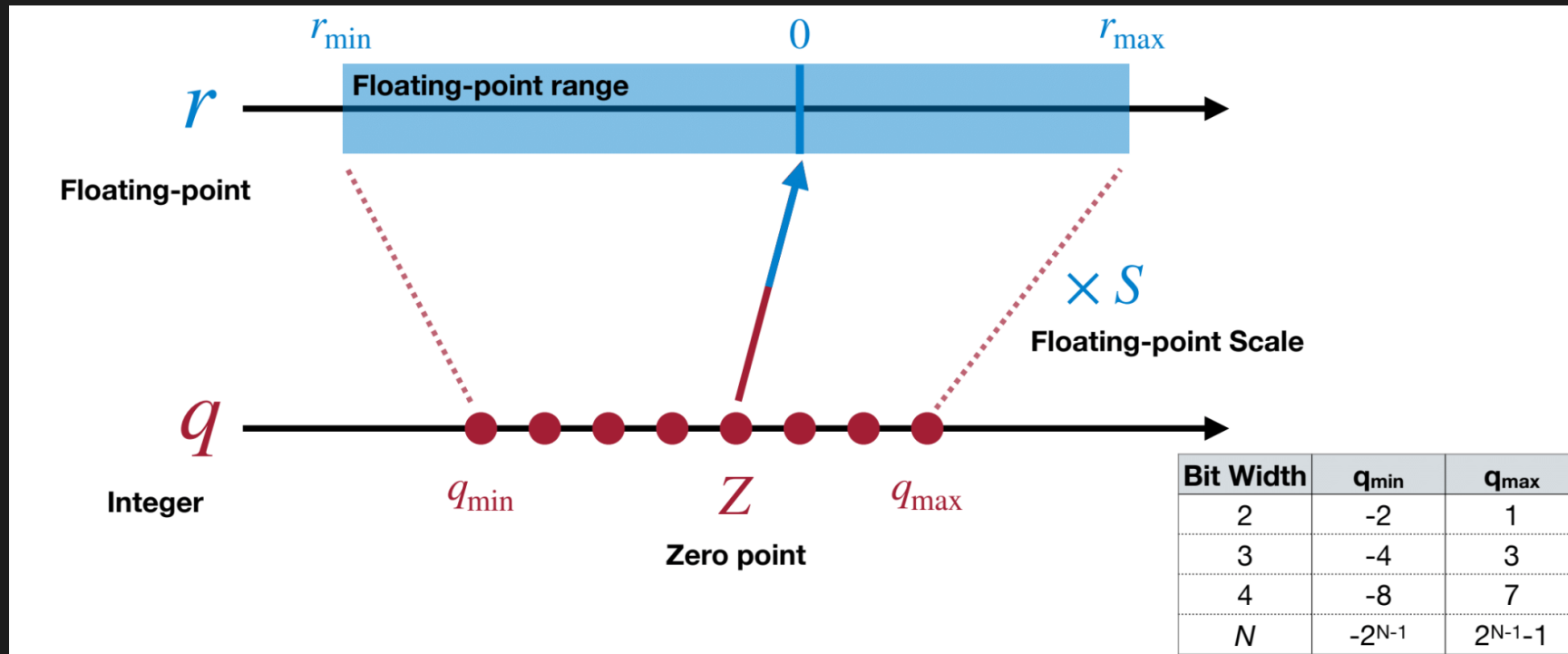
Integer

Floating-point

- quantization parameter
- allow real number $r=0$ be exactly representable by a quantized integer Z
- quantization parameter

How to Perform Quantization

$$r = (q - Z) \times S$$



How to Perform Quantization

$$\begin{aligned} r_{\max} &= S (q_{\max} - Z) \\ r_{\min} &= S (q_{\min} - Z) \end{aligned}$$

$$\downarrow$$
$$r_{\max} - r_{\min} = S (q_{\max} - q_{\min})$$

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

$$r_{\min} = S (q_{\min} - Z)$$

↓

$$Z = q_{\min} - \frac{r_{\min}}{S}$$

↓

$$Z = \text{round} \left(q_{\min} - \frac{r_{\min}}{S} \right)$$

How to Perform Quantization

$$\mathbf{Y} = \mathbf{W}\mathbf{X}$$

$$S_Y (\mathbf{q}_Y - Z_Y) = S_W (\mathbf{q}_W - Z_W) \cdot S_X (\mathbf{q}_X - Z_X)$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} (\mathbf{q}_W - Z_W) (\mathbf{q}_X - Z_X) + Z_Y$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} (\mathbf{q}_W \mathbf{q}_X - Z_W \mathbf{q}_X - Z_X \mathbf{q}_W + Z_W Z_X) + Z_Y$$

How to Perform Quantization

$$Y = WX$$

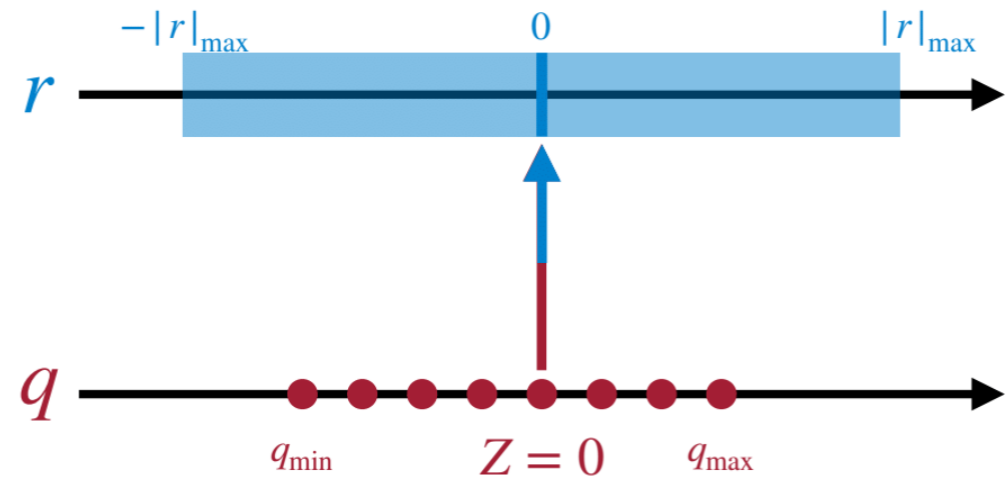
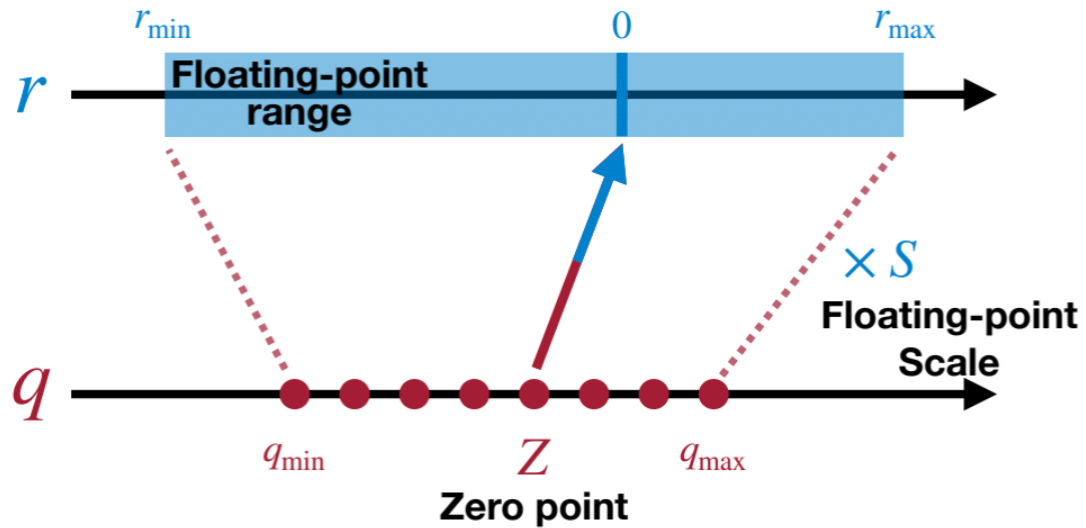
$$q_Y = \frac{S_W S_X}{S_Y} (q_W q_X - Z_W q_X - Z_X q_W + Z_W Z_X) + Z_Y$$

N-bit Integer Multiplication
N-bit Integer Addition

32-bit Integer Addition/Subtraction

Symmetric Quantization

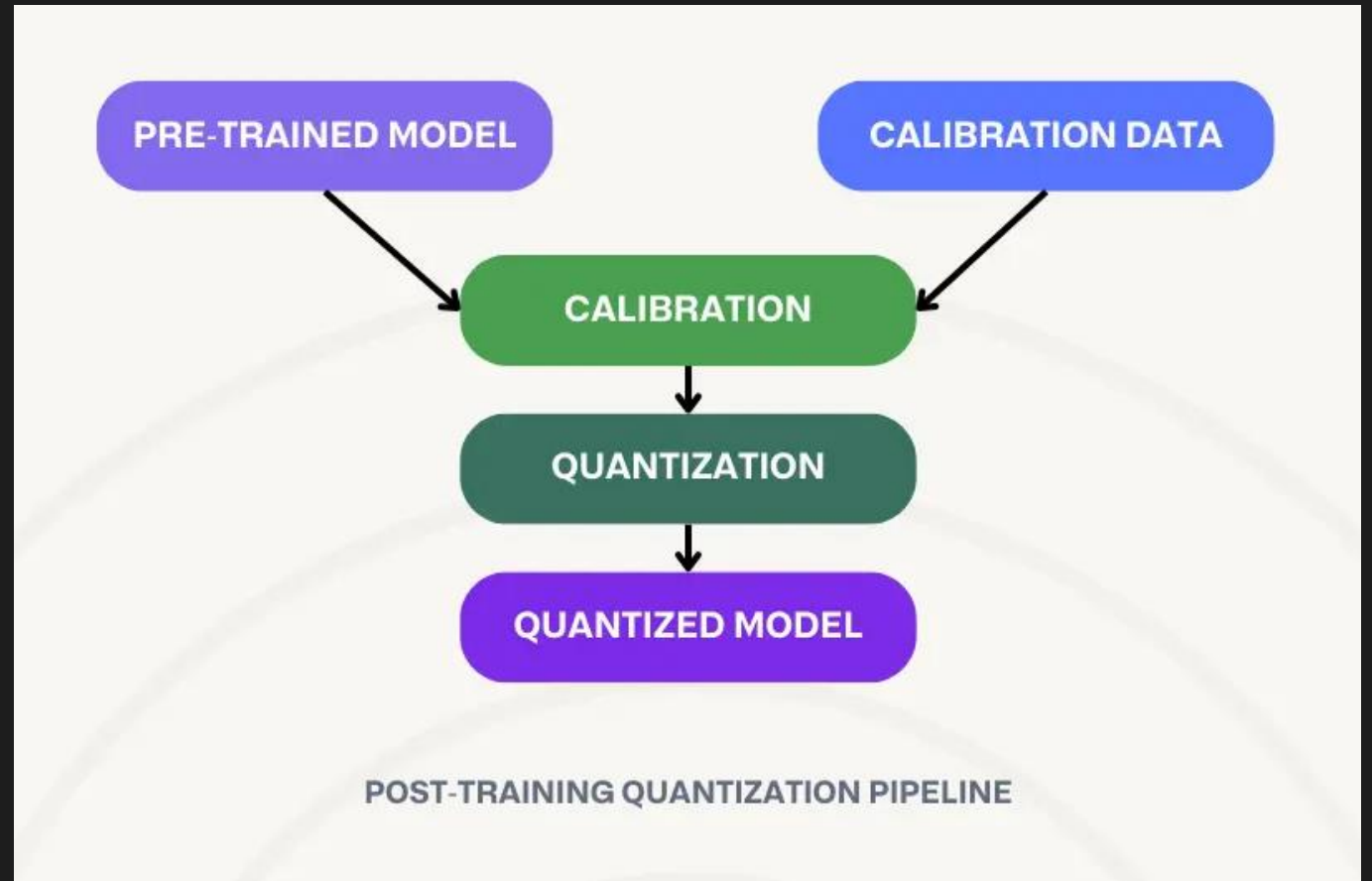
Zero point $Z = 0$ and Symmetric floating-point range



PTQ & QAT

Post Training Quantization

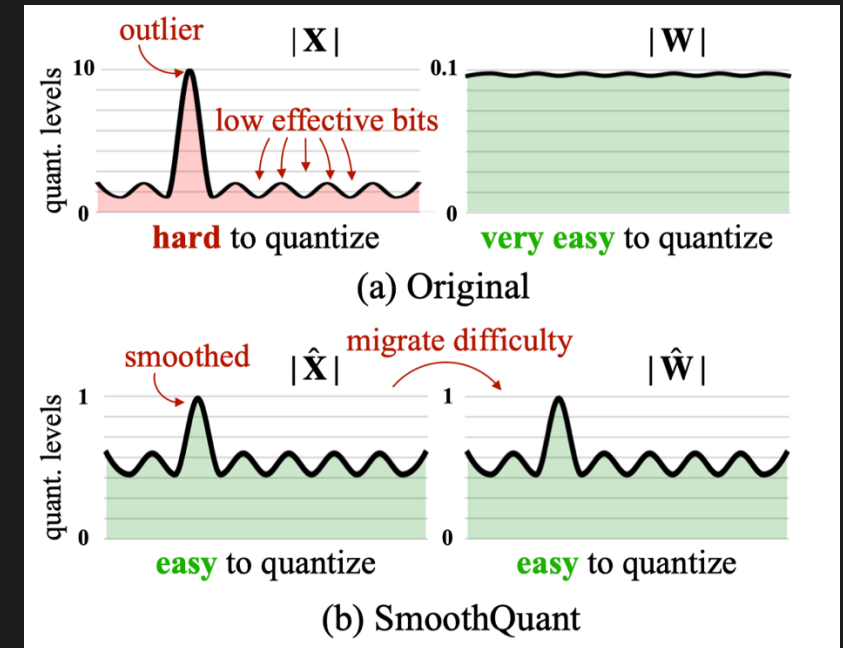
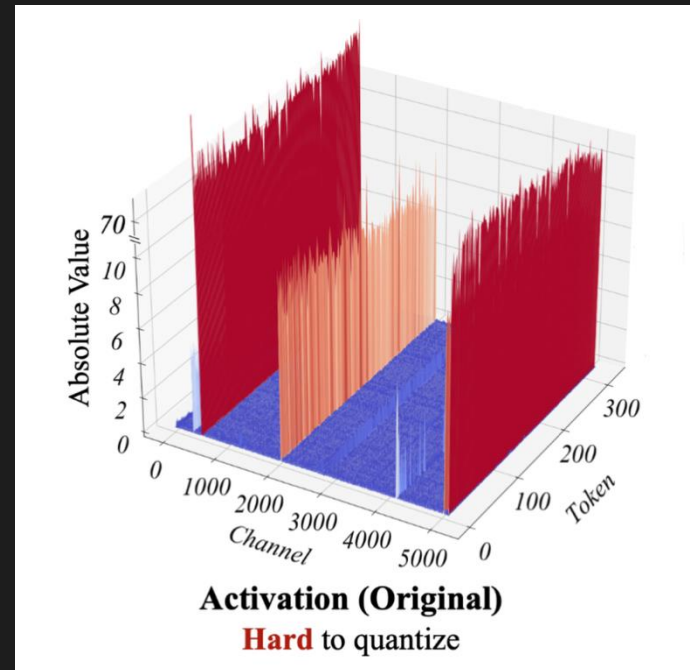
- Compress a pre-trained model at little cost
 - GPTQ
 - AWQ



PTQ & QAT

Post Training Quantization

- Outlier Problem
 - Range Clipping
 - mix-precision
 - SmoothQuant



PTQ & QAT

Quantization-Aware Training

Train the model taking quantization into consideration

- A full precision copy of the weights is maintained throughout the training.
- The small gradients are accumulated without loss of precision.
- Once the model is trained, only the quantized weights are used for inference.

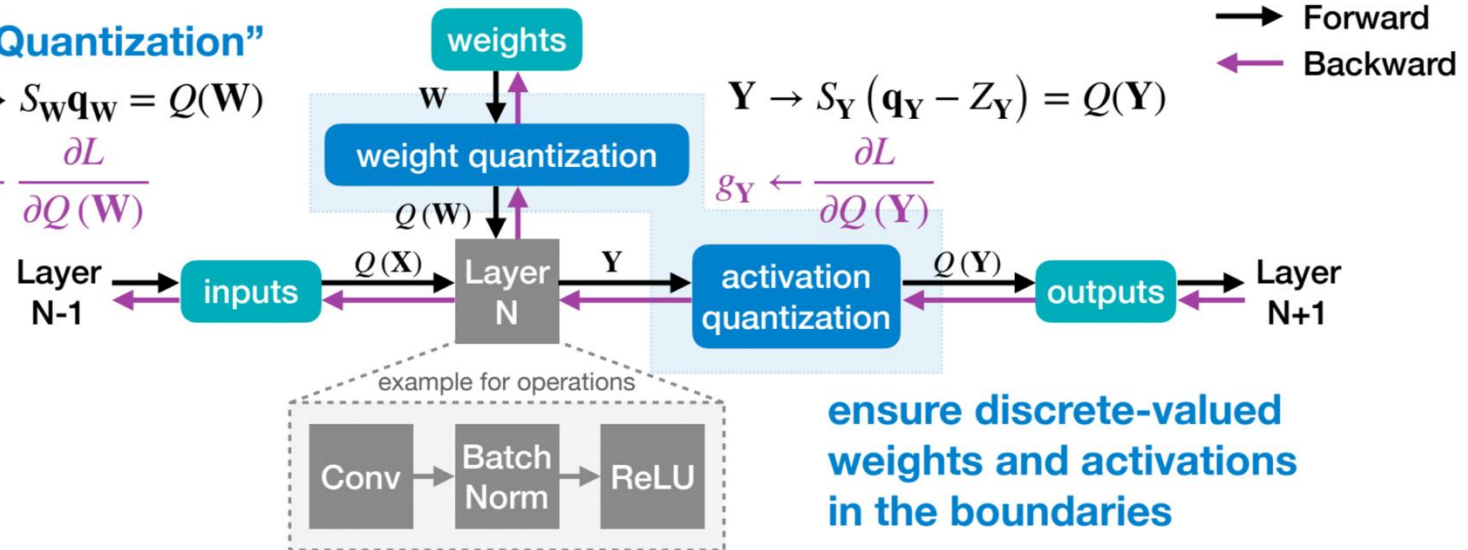
“Simulated/Fake Quantization”

$$W \rightarrow S_W q_W = Q(W)$$

$$g_W \leftarrow \frac{\partial L}{\partial Q(W)}$$

$$Y \rightarrow S_Y (q_Y - Z_Y) = Q(Y)$$

$$g_Y \leftarrow \frac{\partial L}{\partial Q(Y)}$$



ensure discrete-valued weights and activations in the boundaries

these operations still run in full precision

Any Questions?

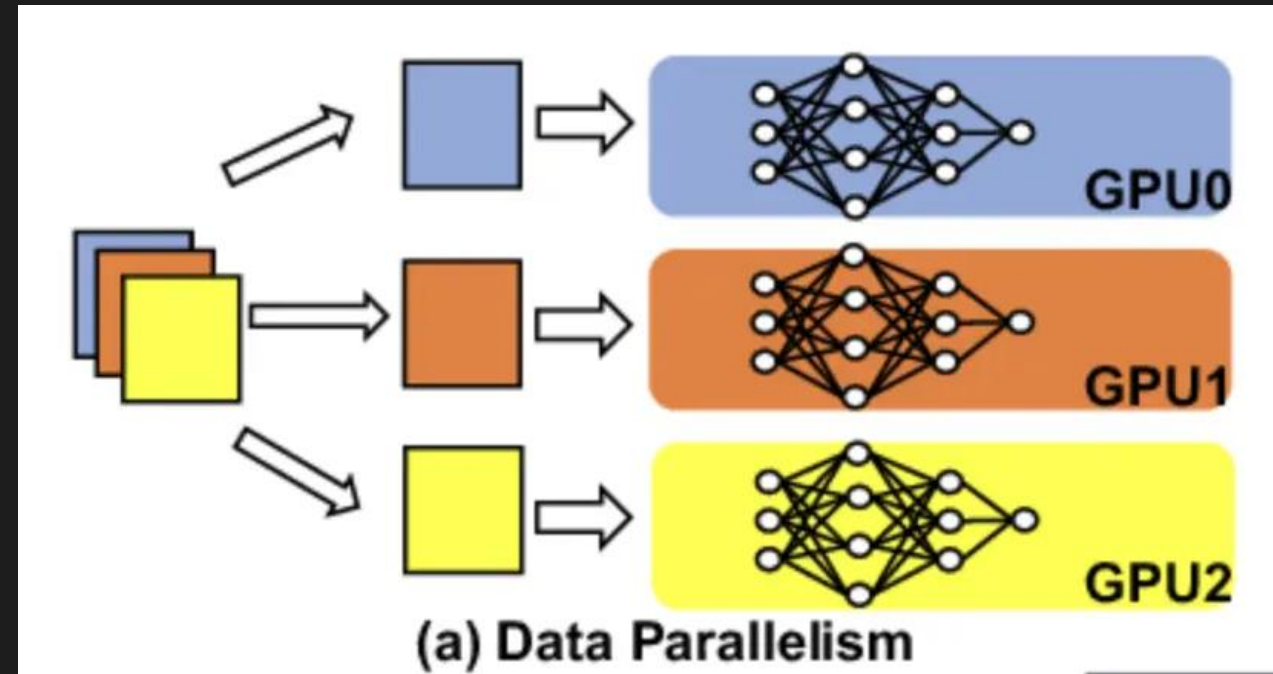
Parallelism

Parallelism

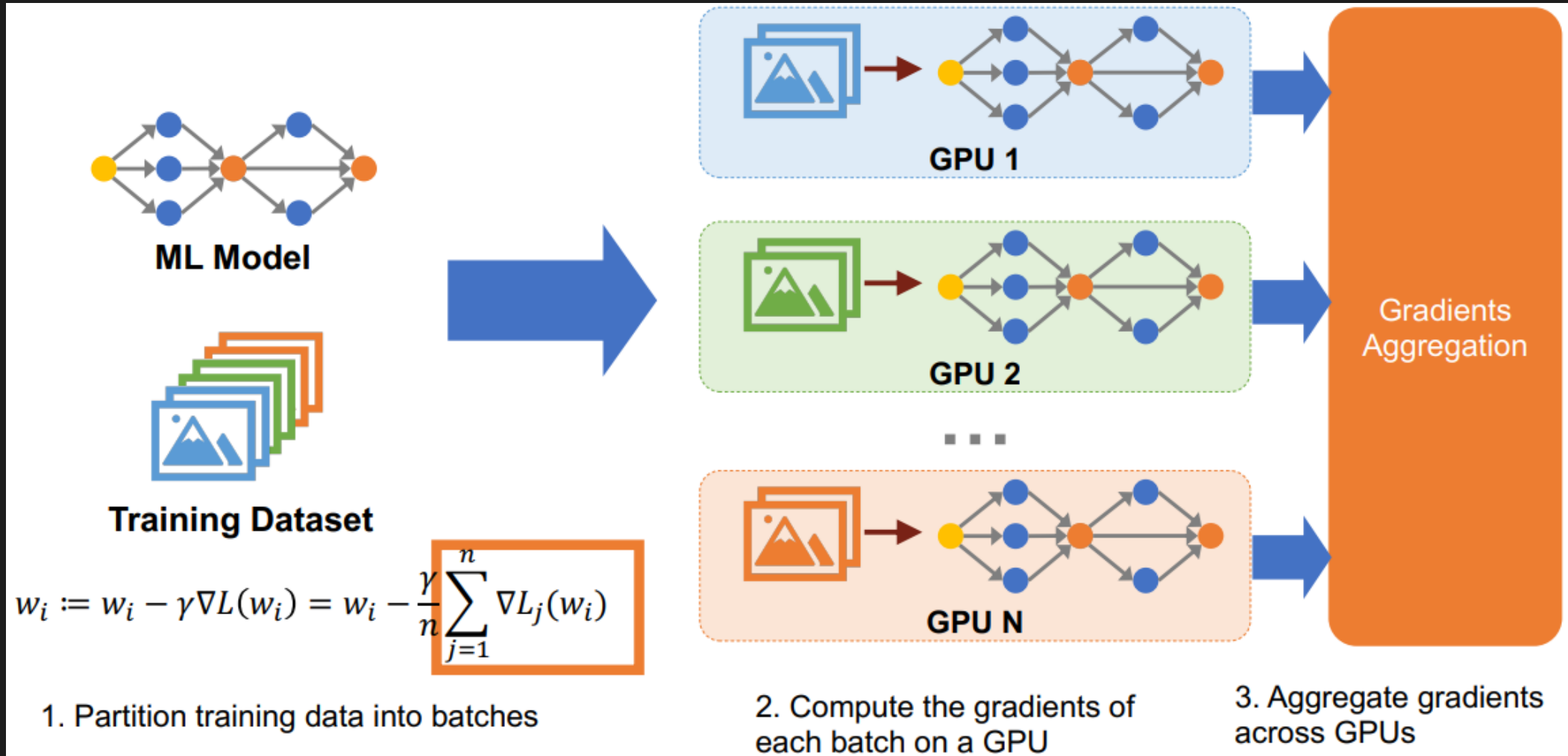
- Data Parallelism
- Pipeline Parallelism
- Tensor Parallelism
- Sequence/Context Parallelism
- Expert Parallelism

Data Parallelism

- Memory Usage: Full model in every device
- Communication: Only before and after inference
- Throughput: Linear Scalability
- Latency: Unaffected

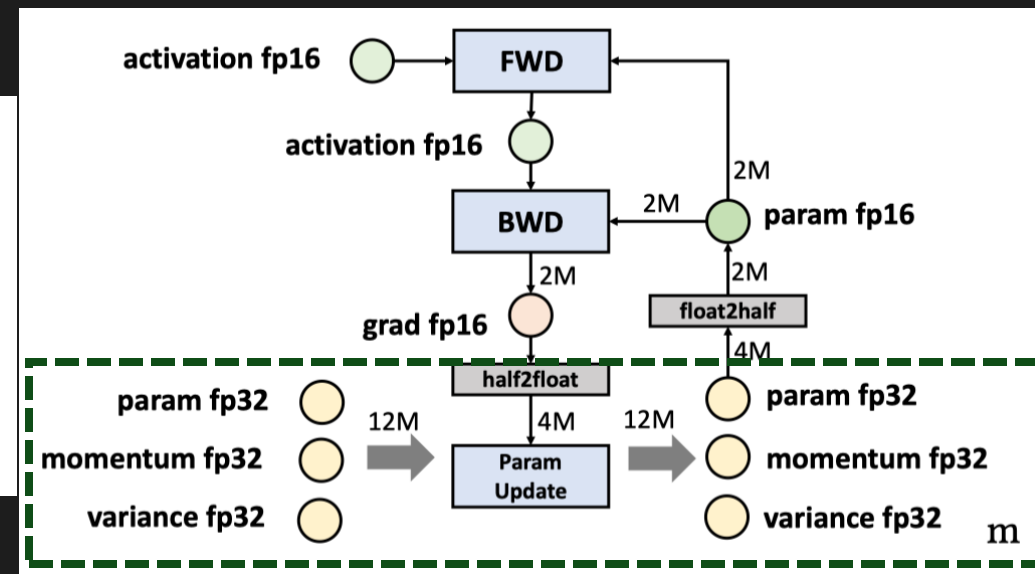
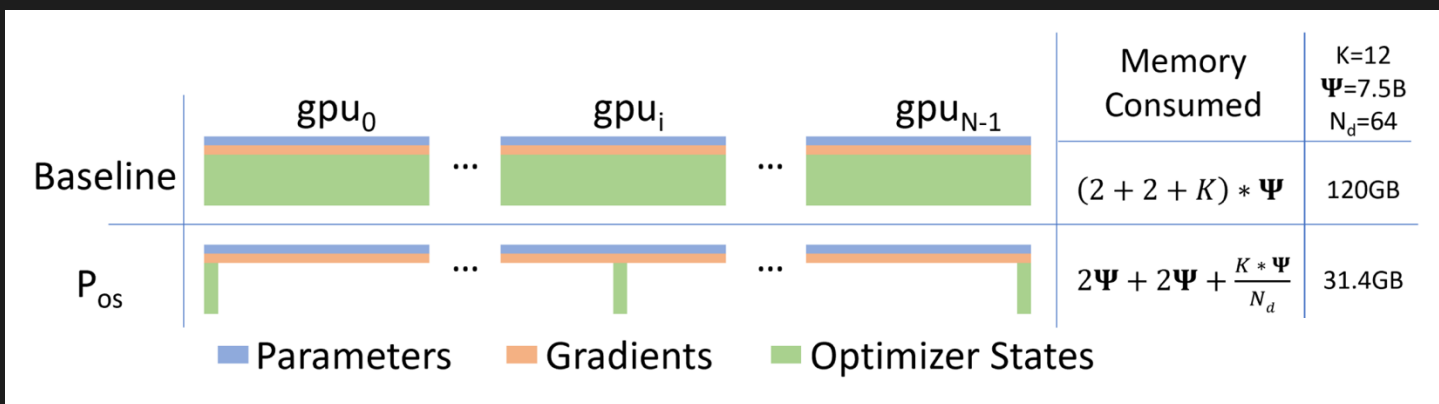


Data Parallelism

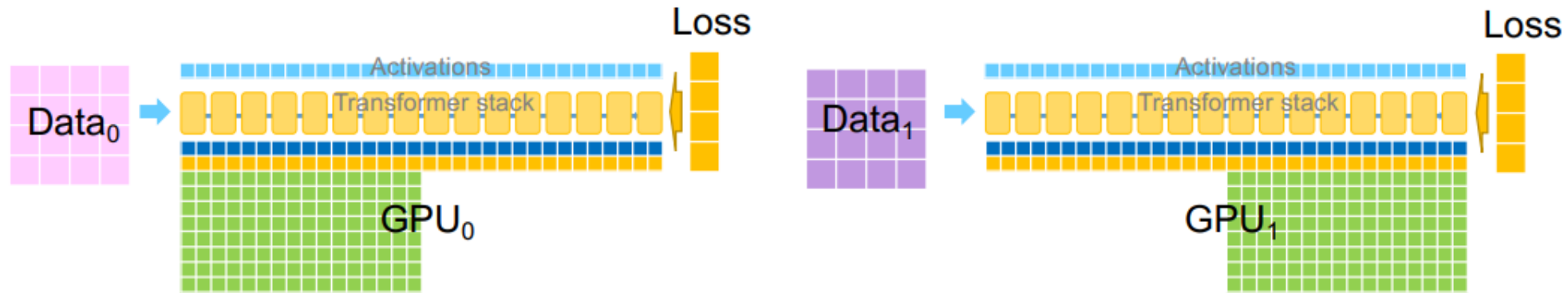


ZeRO

• Stage 1



ZeRO

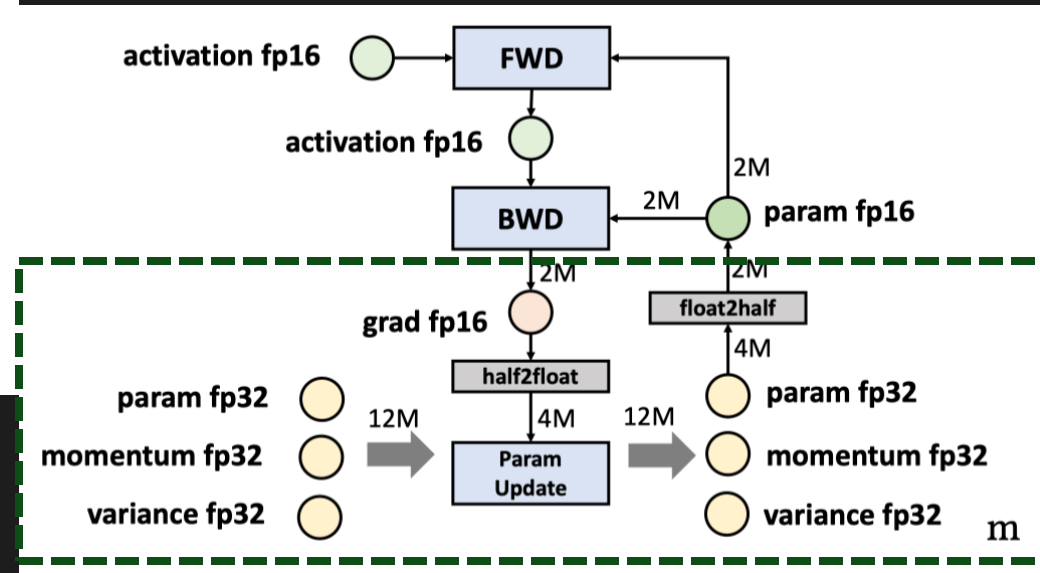


- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients and AllReduce to average
- Update the FP32 weights with ADAM optimizer
- Update the FP16 weights
- All Gather the FP16 weights to complete the iteration

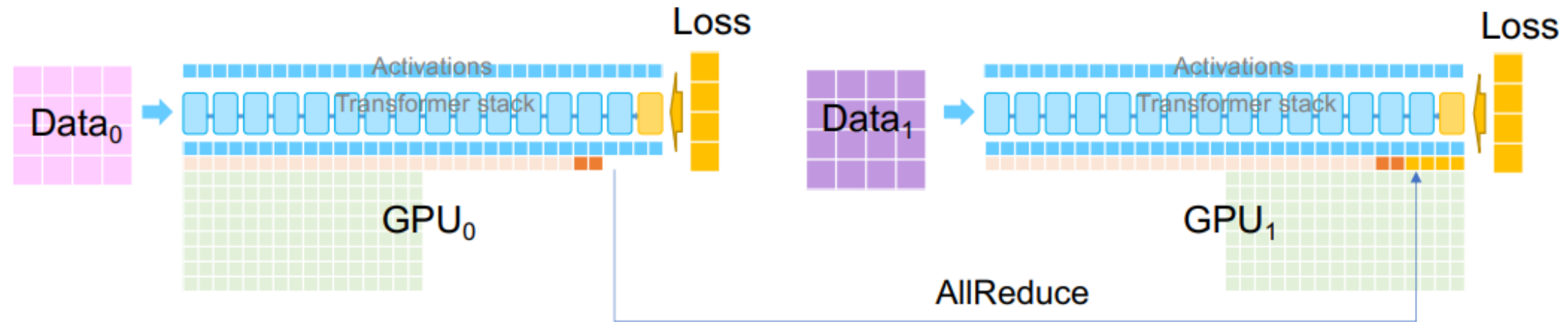
ZeRO

• Stage 2

	gpu ₀	...	gpu _i	...	gpu _{N-1}	Memory Consumed	K=12 Ψ=7.5B N _d =64
Baseline		...		...		$(2 + 2 + K) * \Psi$	120GB
P _{os}		...		...		$2\Psi + 2\Psi + \frac{K * \Psi}{N_d}$	31.4GB
P _{os+g}		...		...		$2\Psi + \frac{(2+K)*\Psi}{N_d}$	16.6GB



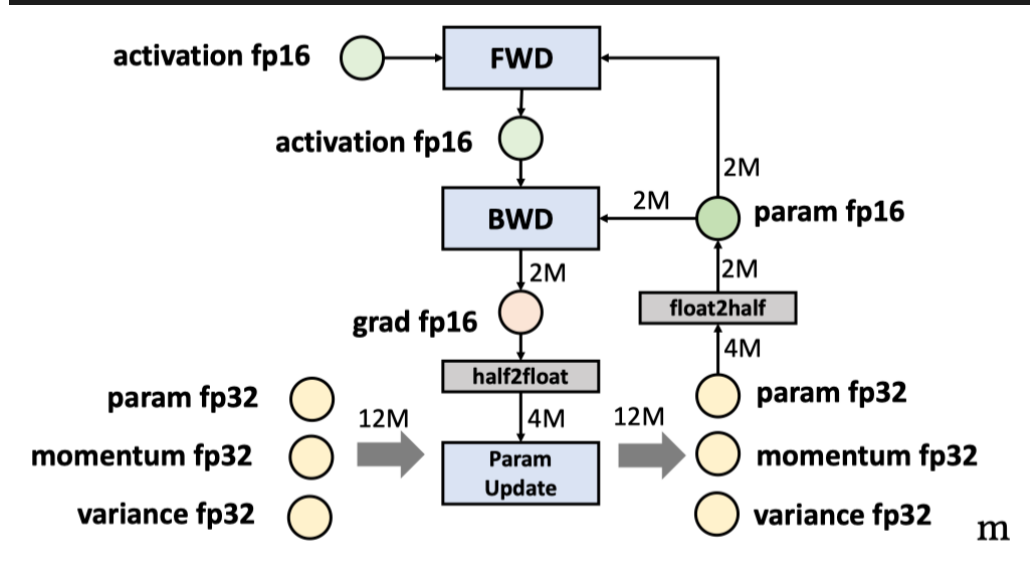
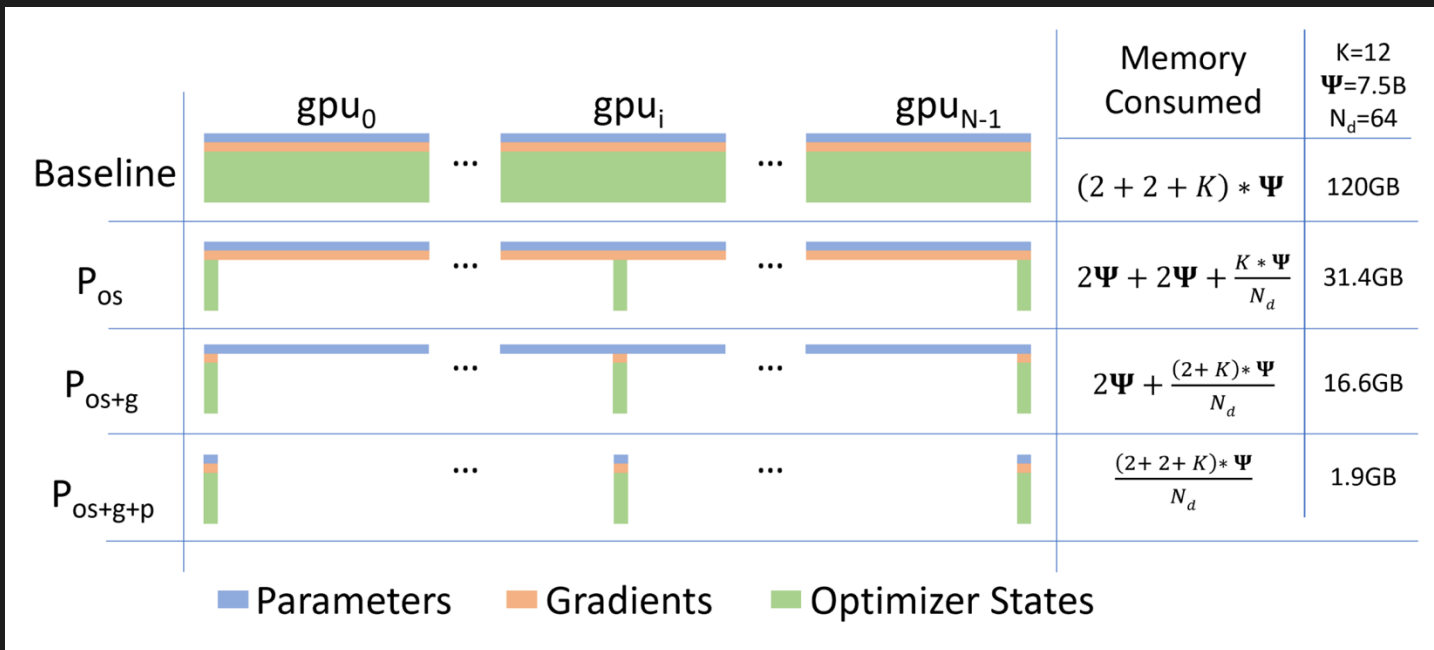
ZeRO



- Partitioning gradients across GPUs
- Reduce gradients on GPUs responsible for updating parameters

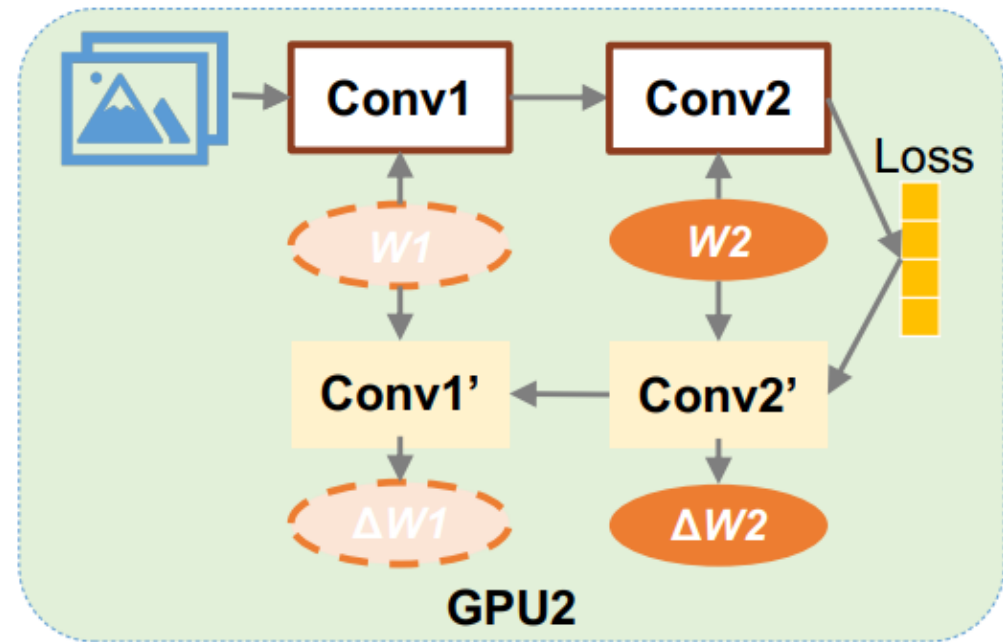
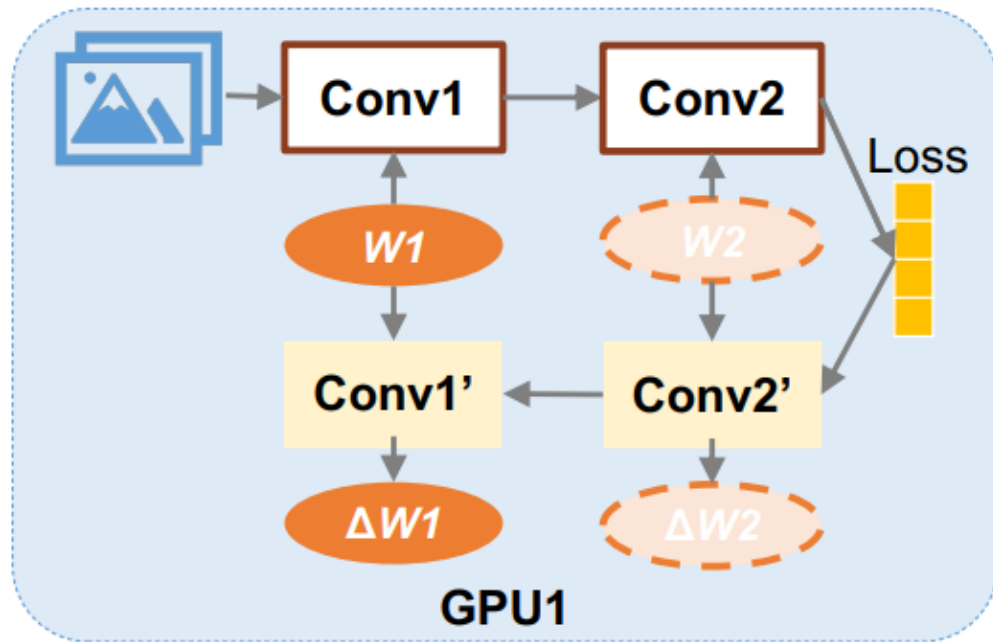
ZeRO

• Stage 3



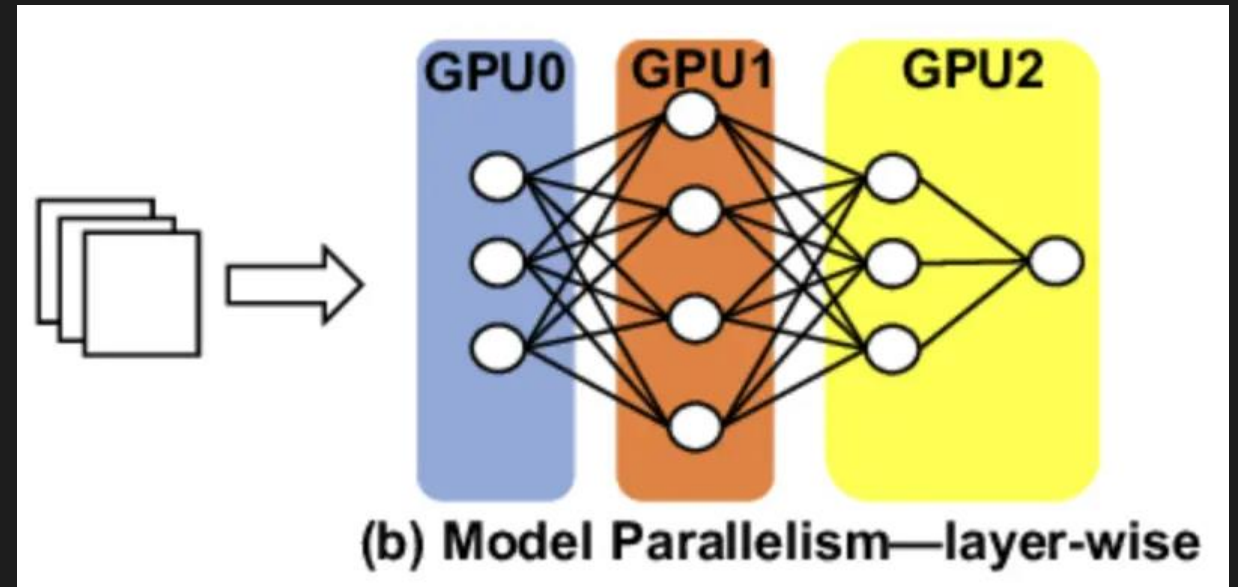
ZeRO

- In ZeRO, model parameters are partitioned across GPUs
- GPUs broadcast their parameters again during backward



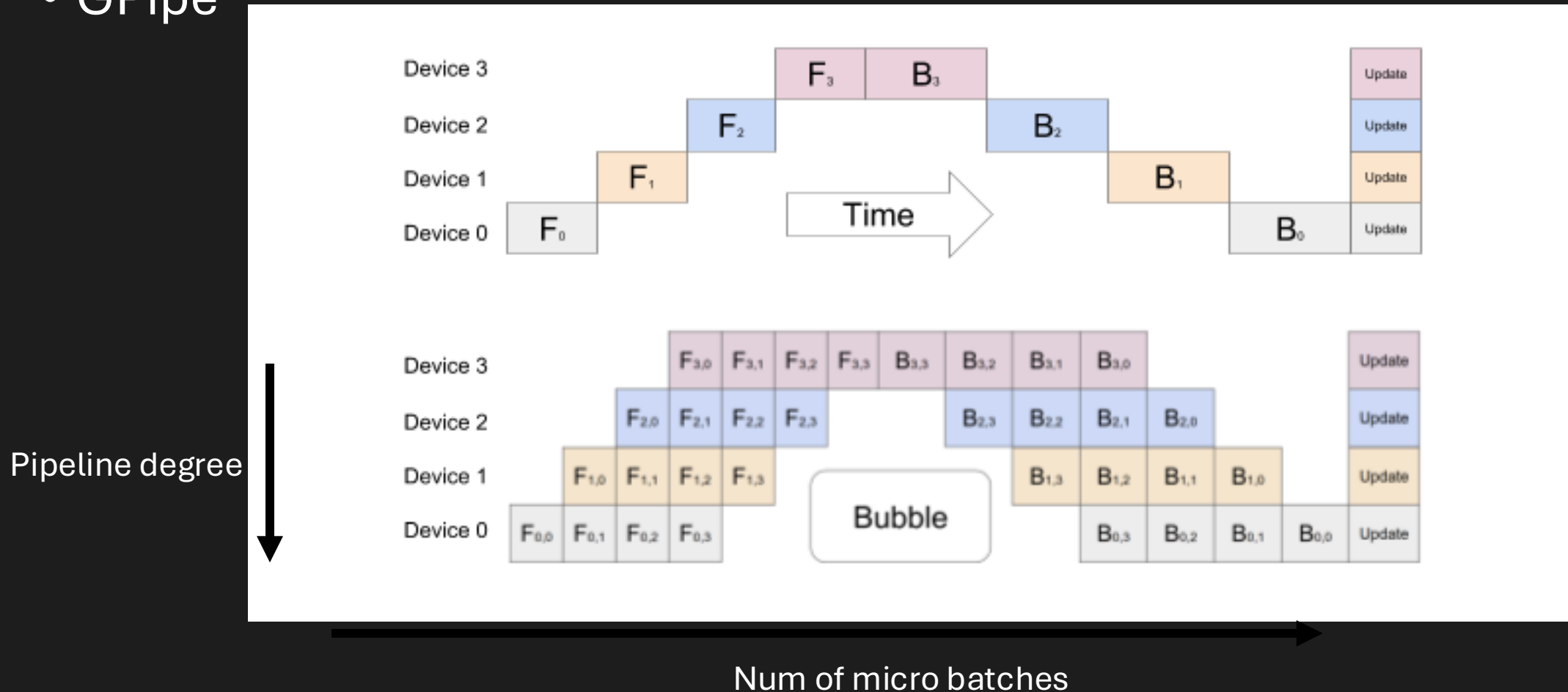
Pipeline Parallelism

- Memory Usage: Separate model by layer among devices
- Communication: After every stage
- Throughput: Scalability
- Latency: Delayed



Pipeline Parallelism

- GPipe

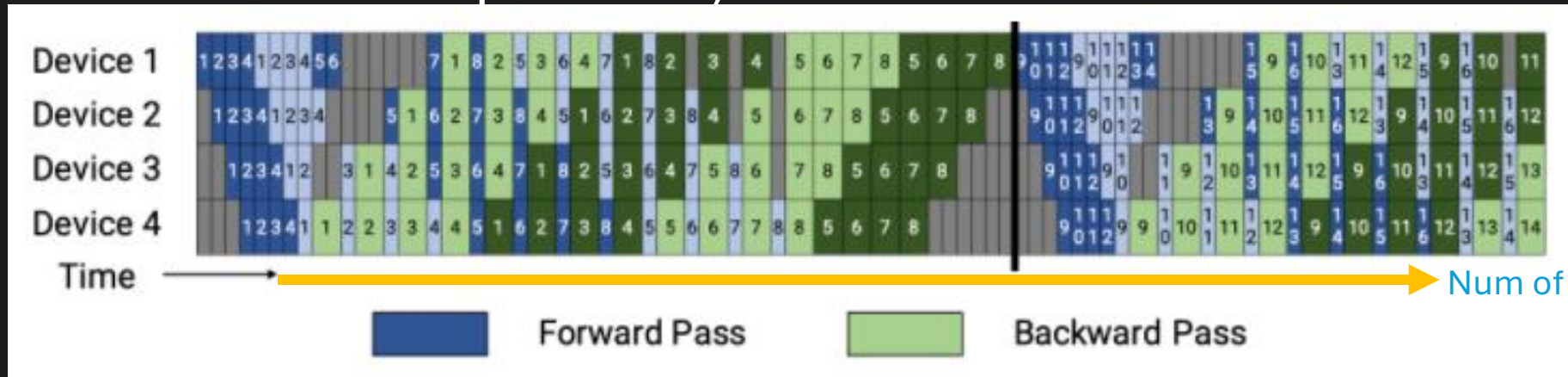


Pipeline Parallelism

- 1F1B (one forward pass followed by one backward pass)

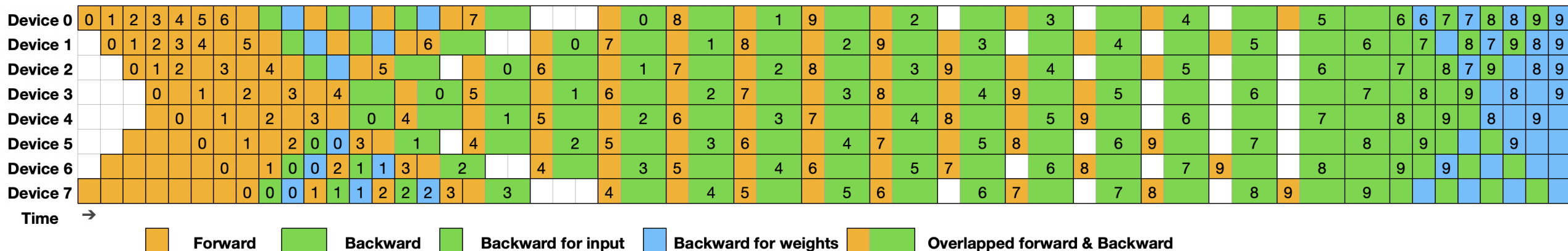


- Interleaved pipeline (backward execution of the microbatches is prioritized whenever possible)



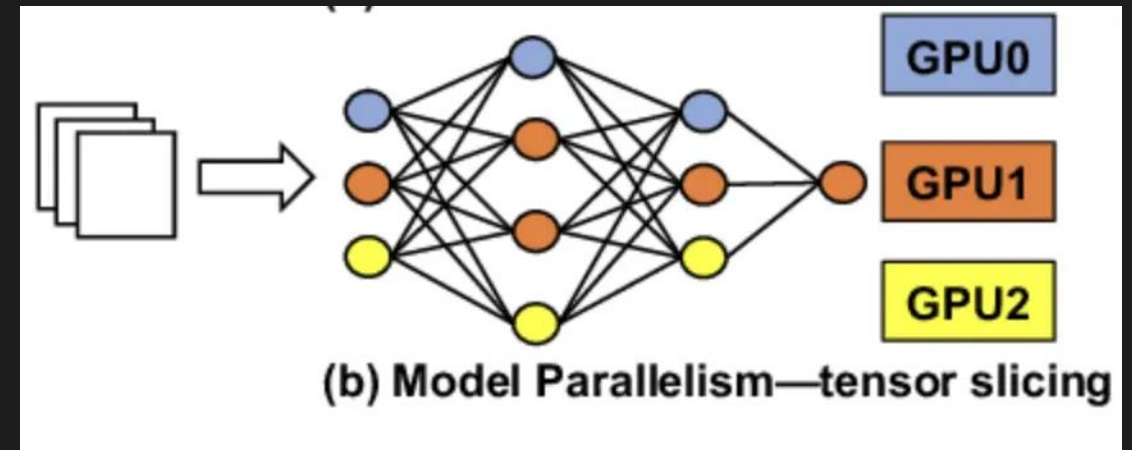
Pipeline Parallelism

DualPipe (DeepSeek)



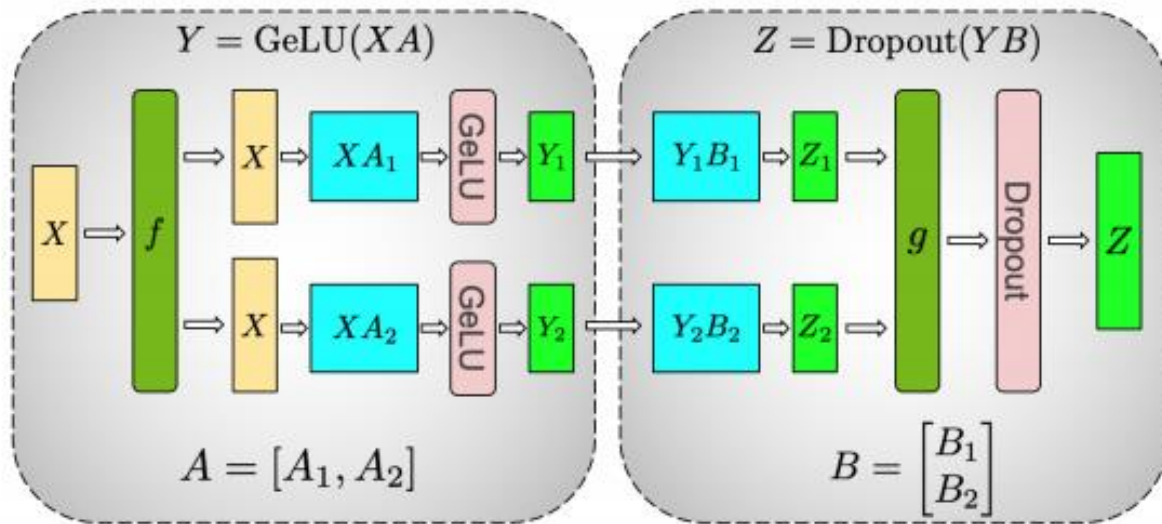
Tensor Parallelism

- Memory Usage: Separate model by tensor among devices
- Communication: in almost every operation
- Throughput: Scalability
- Latency: Communication-bound

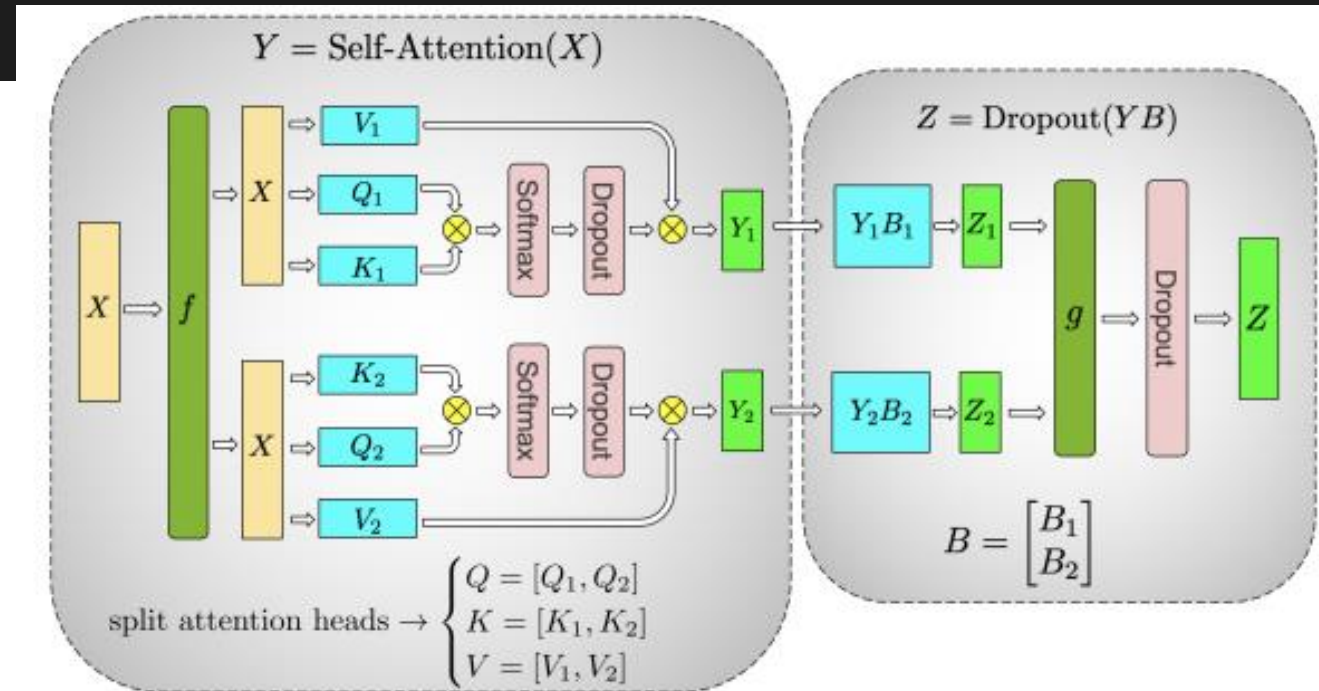


Tensor Parallelism

- 1D
 - Proposed first in Megatron



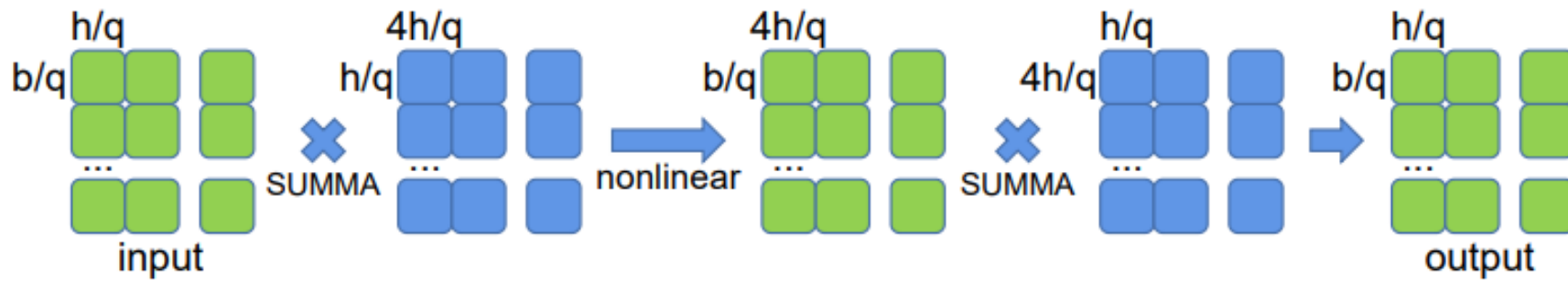
(a) MLP



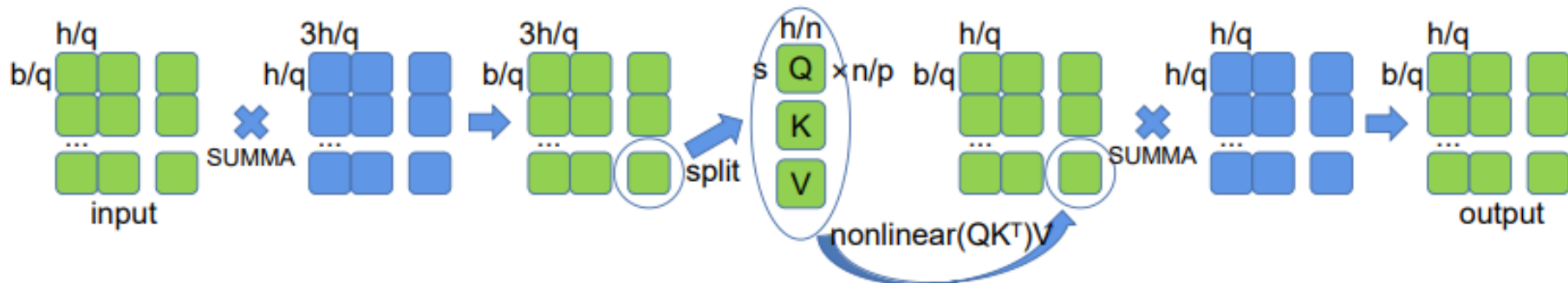
(b) Self-Attention

Tensor Parallel

- 2D
 - An Efficient 2D Method for Training Super-Large Deep Learning Models



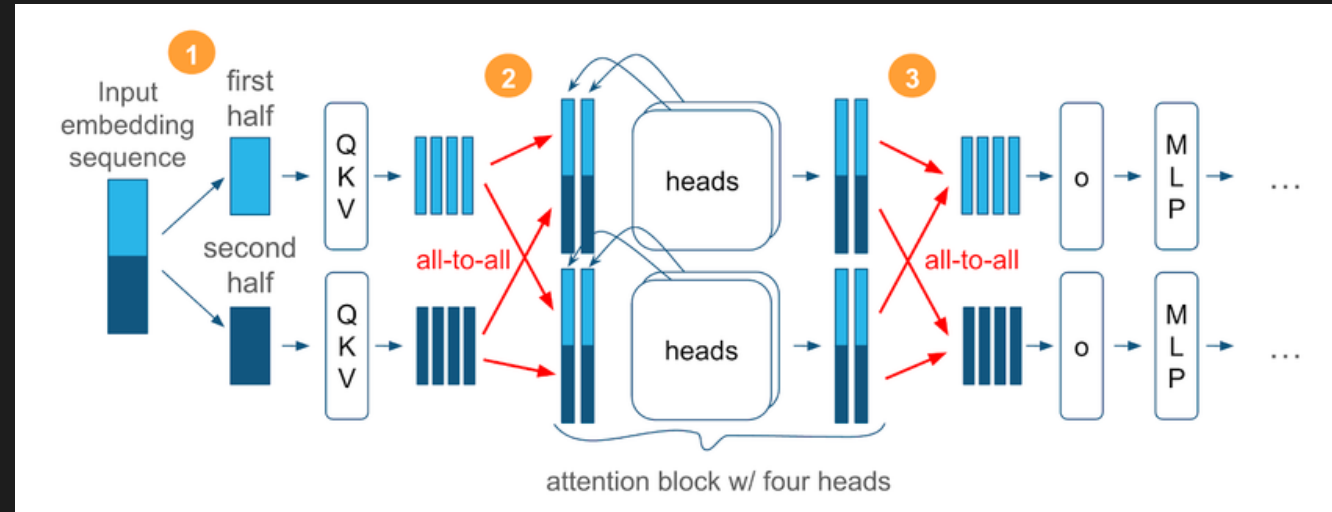
(a) MLP



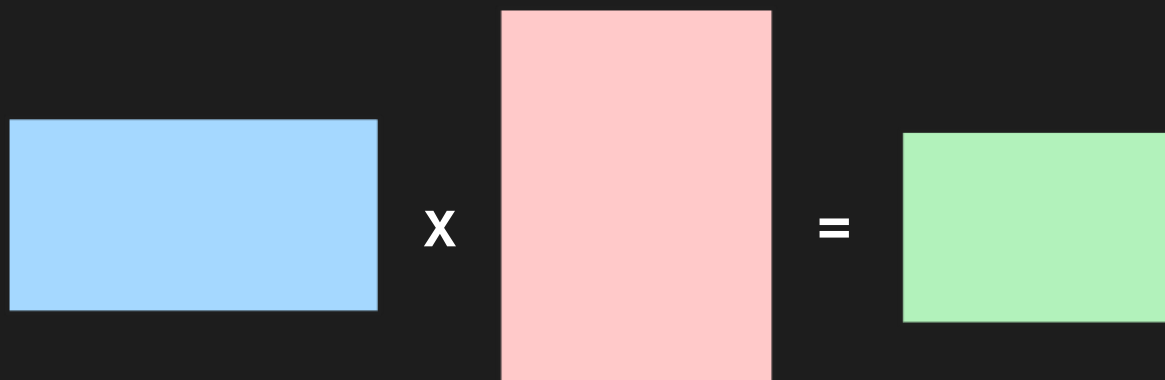
(b) self-attention

Sequence/Context Parallelism

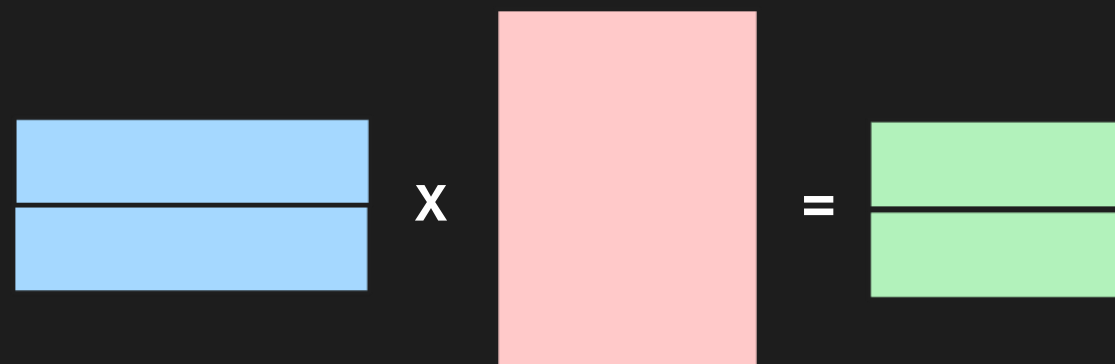
- Memory Usage: Sharded activation
- Communication: After some operations
- Throughput: Scalability
- Latency: Communication-bound



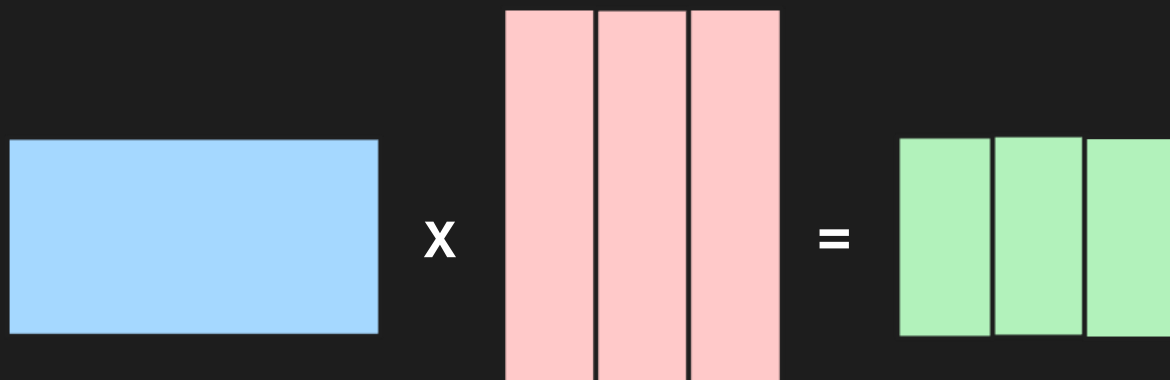
Sequence/Context Parallelism



No parallelism



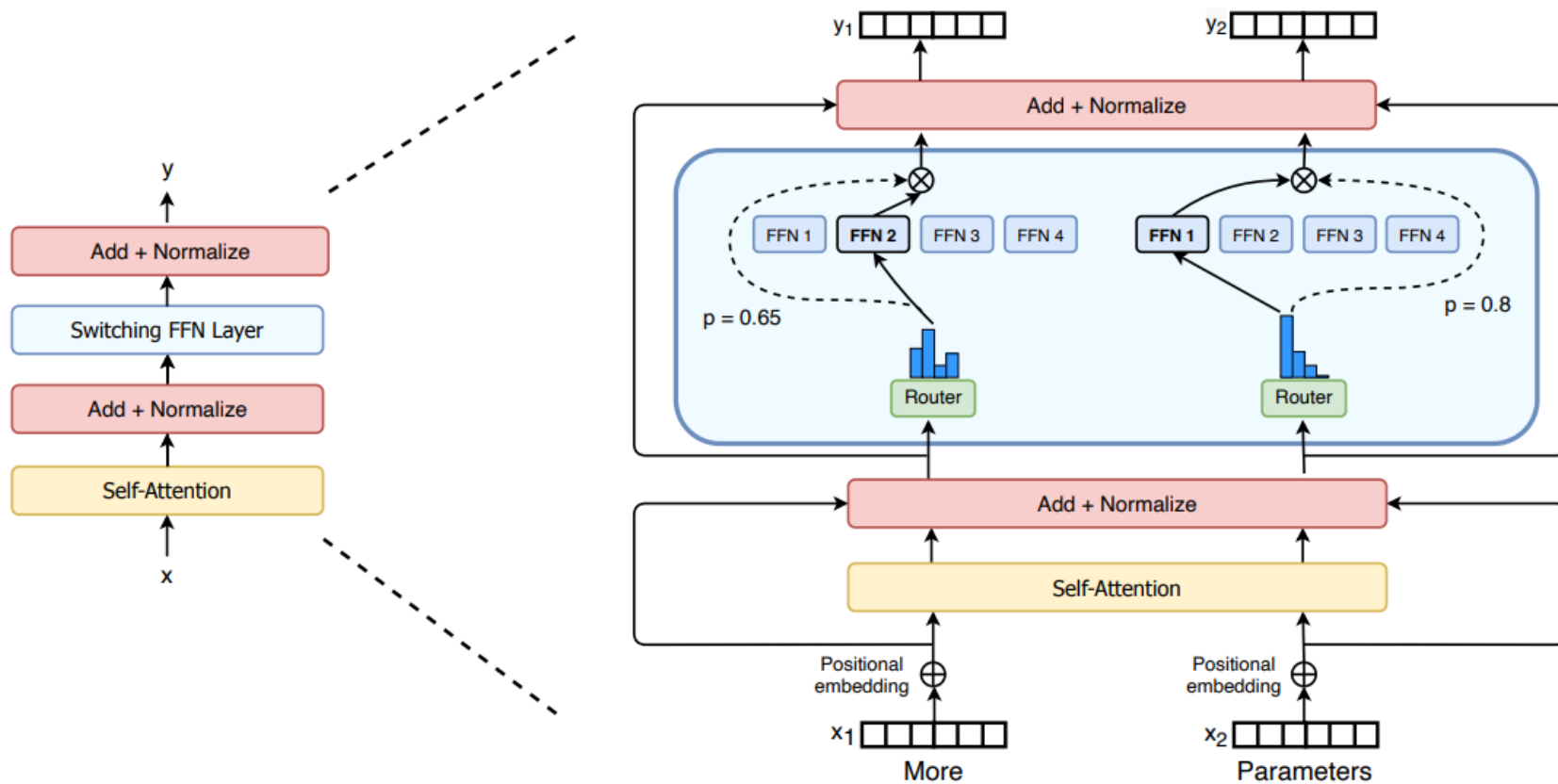
Sequence Parallelism



Tensor Parallelism

Expert Parallelism

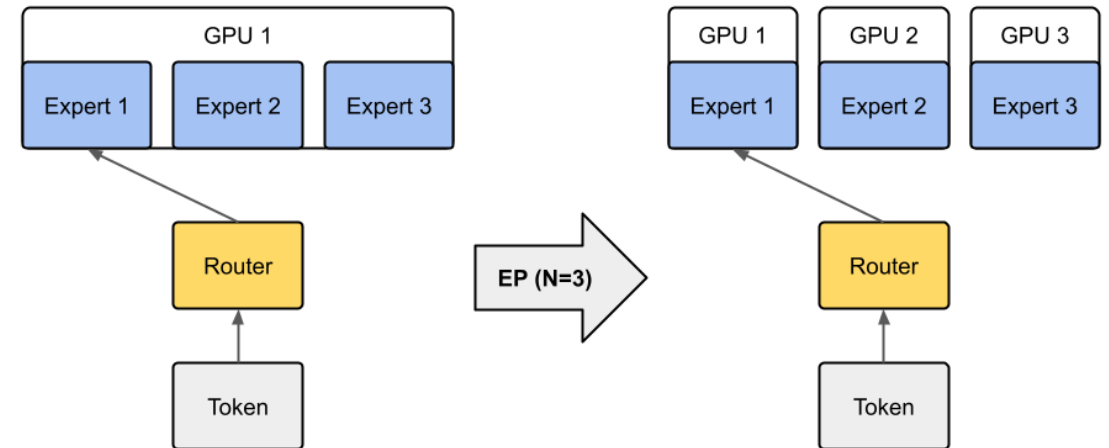
Mixture of Expert



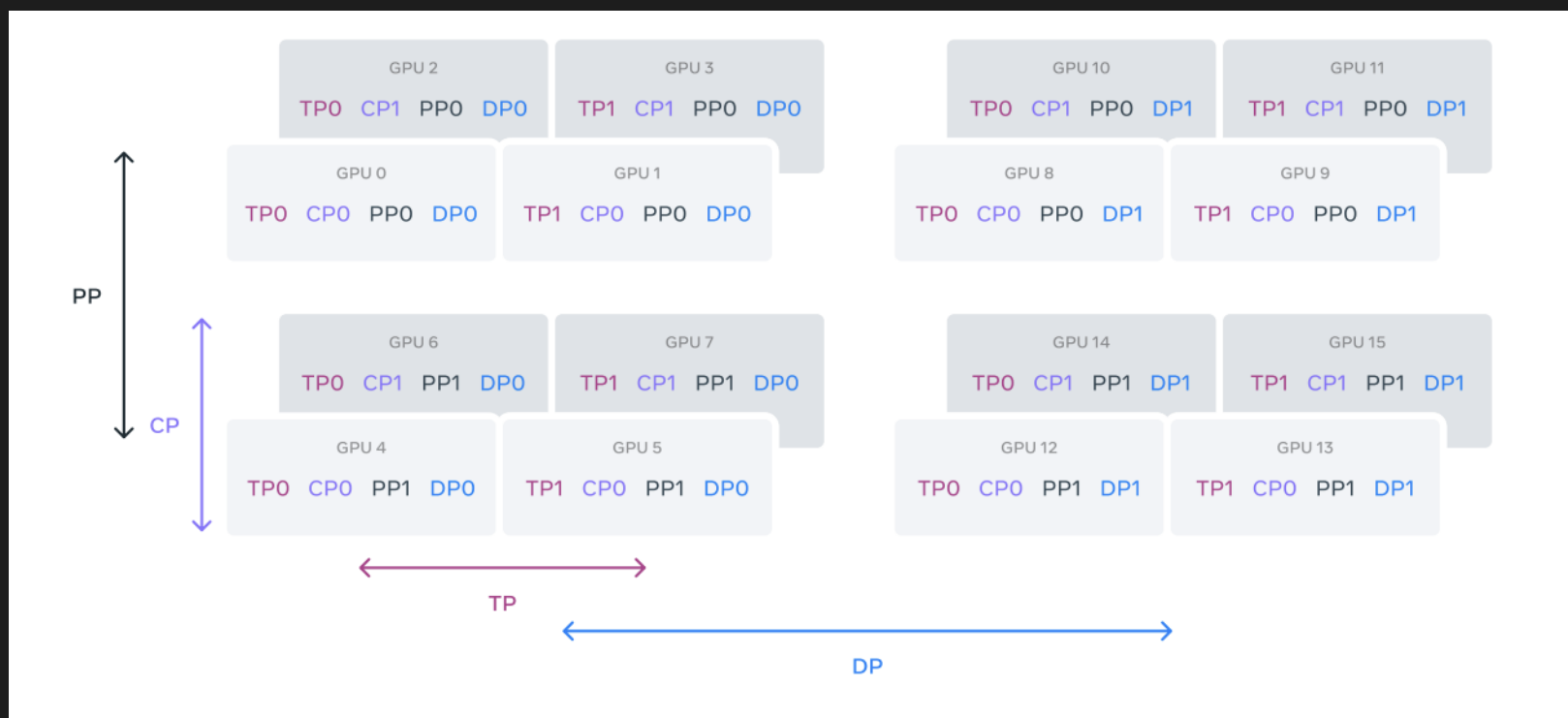
Expert Parallelism

- Memory Usage: Sharded expert parameters
- Communication: After expert FFN

Expert Parallelism applied on Mixture-of-Experts.



Mixed Parallelism



4D Parallelism in Llama 3

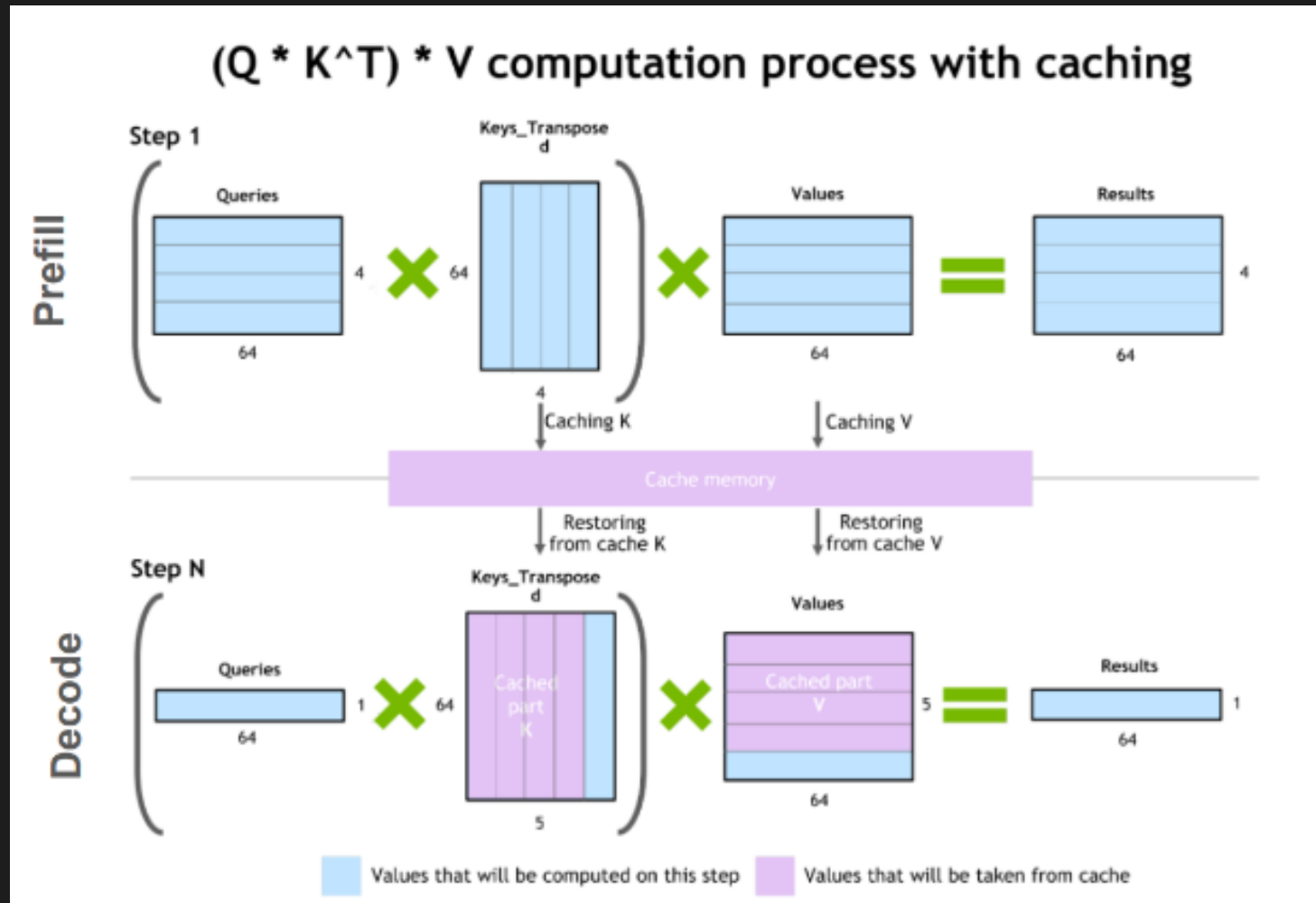
Any Questions?

Additional Optimization Topics

Efficient Training Procedure

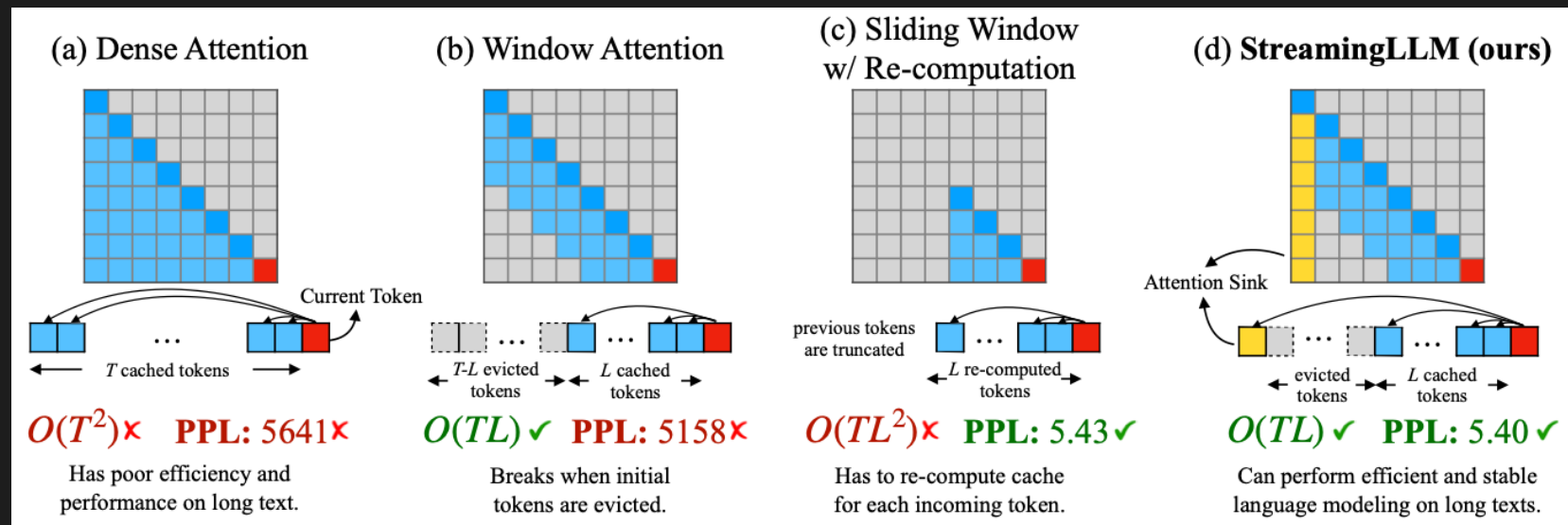
- Gradient Accumulation
 - Increase batch size
 - Reduce relative GPU memory cost
 - Reduce overall optimizer iteration steps
- Gradient Checkpointing
 - Release activation records after forward
 - Slow down 20~30%, but reduce memory footprints

KV Cache

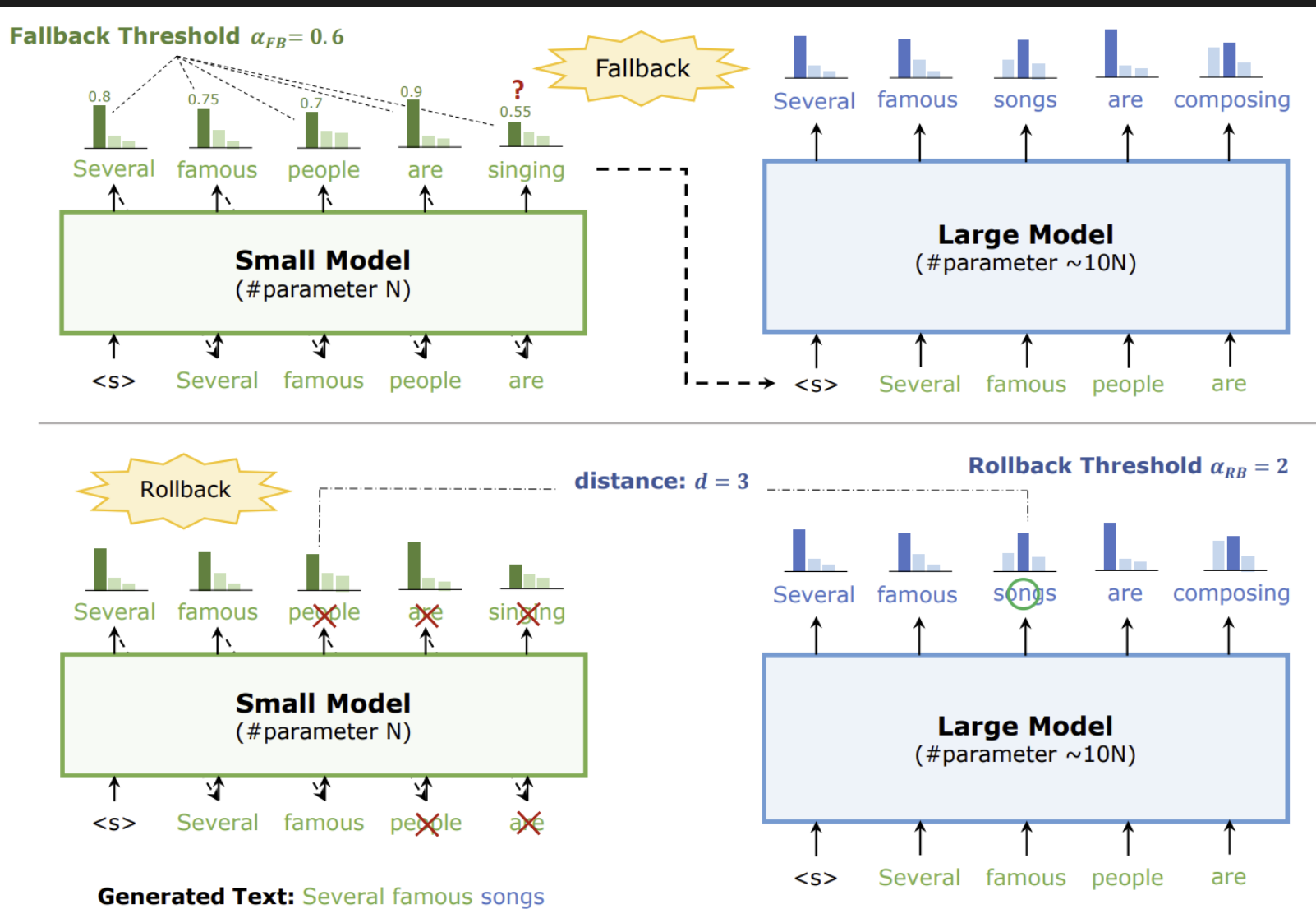


KV Cache Compression

- Evicting tokens in KV Cache
- Merging KV Cache between layers
- KV Cache Quantization



Speculative Decoding



Continuous Batching

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3	S_3				
S_4	S_4	S_4	S_4	S_4			

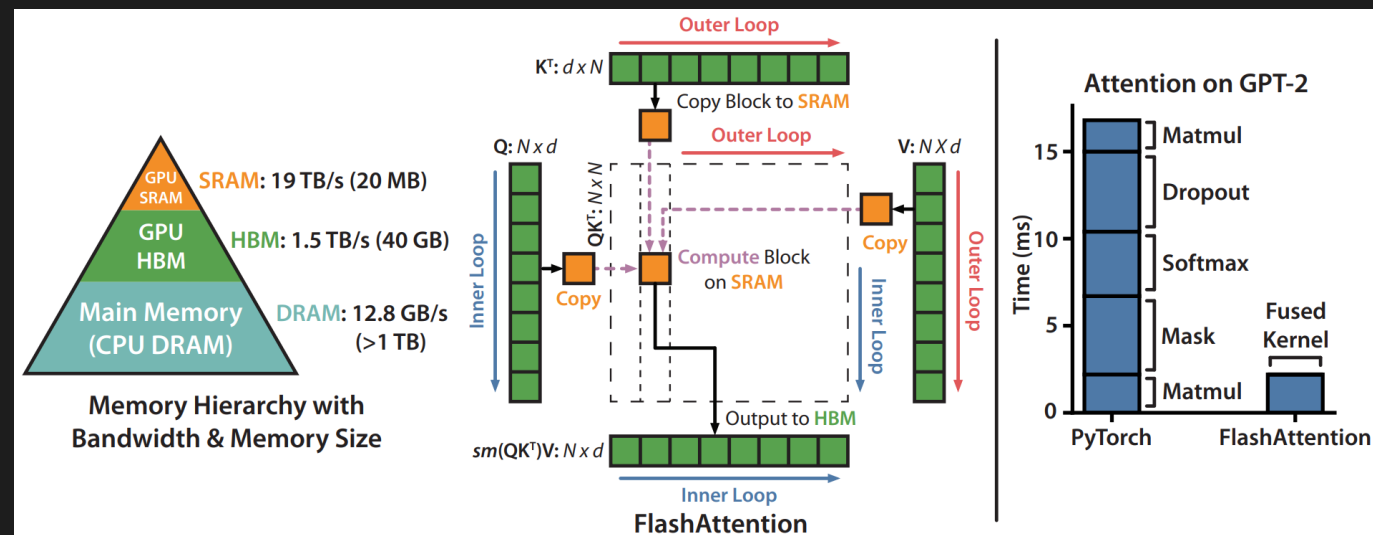
T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

Efficient Operator

- Operator Fusion



- More efficient operator
 - FlashAttention



Any Questions?

References & Further Studying

- **Deep Learning System Course**

- <https://dlsyscourse.org/>

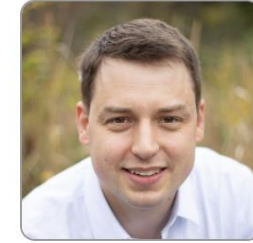
- **TinyML and Efficient Deep Learning Computing**

- <https://efficientml.ai>

Instructors



Tianqi Chen



Zico Kolter

Instructor



Song Han 

Associate Professor

Thanks